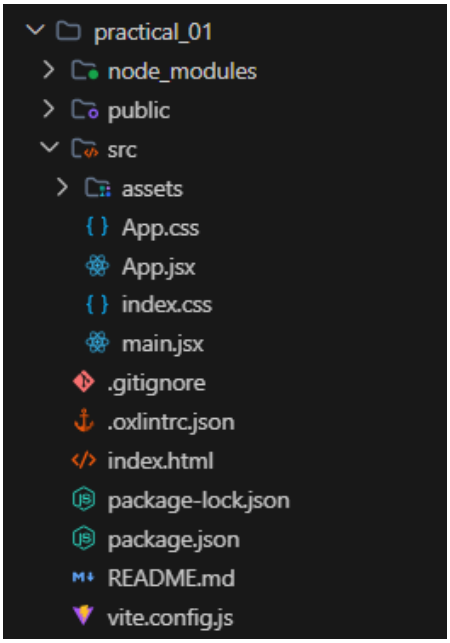
PRACTICAL NO. 01

Build a React Application using Functional Components, JSX and Component Reusability Concepts

Software Requirements

- Visual Studio Code
- Node.js and npm
- React
- Vite

Folder Structure



Source Code

1. App.jsx

```
import './App.css'

function MonumentCard({ name, state, builtBy, era }) {
  return (
    <div className="monument-card">
      <h2>{name}</h2>
      <p><strong>State:</strong> {state}</p>
      <p><strong>Built By:</strong> {builtBy}</p>
      <p><strong>Era:</strong> {era}</p>
    </div>
  )
}

function App() {
  return (
    <div className="app">
      <h1>Historical Monuments of India</h1>

      <div className="card-container">
        <MonumentCard
          name="Qutub Minar"
          state="Delhi"
          builtBy="Qutb-ud-din Aibak"
          era="Delhi Sultanate"
        />

        <MonumentCard
          name="Konark Sun Temple"
          state="Odisha"
          builtBy="King Narasimhadeva I"
          era="Eastern Ganga Dynasty"
        />

        <MonumentCard
          name="Mysore Palace"
          state="Karnataka"
          builtBy="Maharaja Krishnaraja Wadiyar IV"
          era="Wadiyar Dynasty"
        />
      </div>
    </div>
  )
```

```
}

export default App
```

2. App.css

```
#root {
  width: 100%;
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  background-color: #f3f4f6;
  color: #1f2937;
  min-height: 100vh;
}

.app {
  min-height: 100vh;
  padding: 50px 20px;
  text-align: center;
  max-width: 1100px;
  margin: 0 auto;
}

.app h1 {
  margin-bottom: 35px;
  color: #0f766e;
  font-size: 2.2rem;
  font-weight: 700;
}

.card-container {
  display: flex;
  justify-content: center;
  gap: 25px;
  flex-wrap: wrap;
}

.monument-card {
  width: 290px;
  padding: 24px 20px;
  background: #ffffff;
  border-radius: 10px;
  border: 1px solid #e5e7eb;
  border-top: 4px solid #0d9488;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.05);
  text-align: left;
  transition: transform 0.2s ease, box-shadow 0.2s ease;
}

.monument-card:hover {
  transform: translateY(-4px);
  box-shadow: 0 8px 20px rgba(0, 0, 0, 0.08);
}

.monument-card h2 {
  margin-top: 0;
  margin-bottom: 15px;
  font-size: 1.3rem;
  color: #111827;
  text-align: center;
  border-bottom: 1px solid #f3f4f6;
  padding-bottom: 10px;
}

.monument-card p {
  margin: 10px 0;
  font-size: 15px;
  color: #4b5563;
  line-height: 1.5;
}

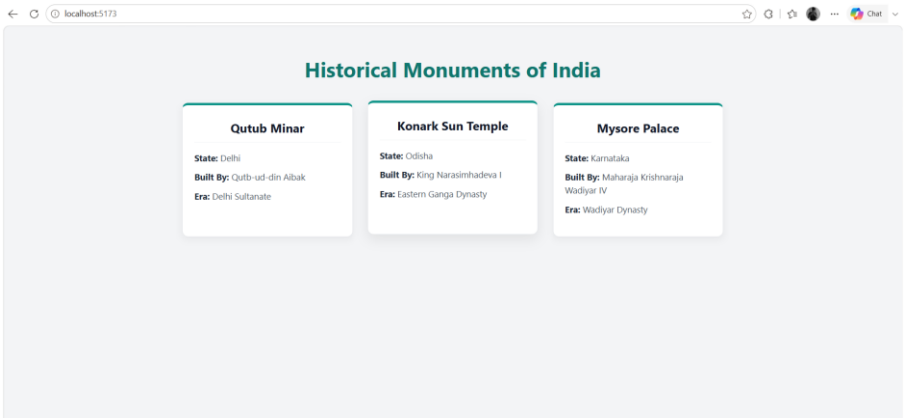
.monument-card p strong {
  color: #1f2937;
}
```

3. main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'
```

```
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Output



Result

Thus, the React application was successfully developed using Functional Components, JSX and Component Reusability concepts.

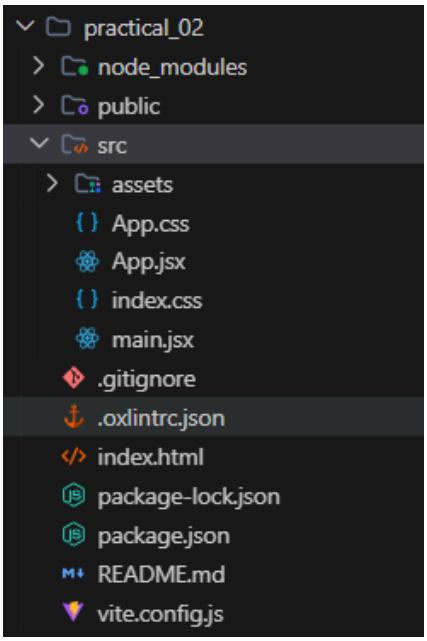
PRACTICAL NO. 02

Develop a React application implementing Props, State Management and Event Handling.

Software Requirements

- Code Editor: Visual Studio Code
- Runtime Environment: Node.js
- Package Manager: npm
- JavaScript Library: React.js
- Build Tool: Vite

Folder Structure



Source Code

1. App.jsx

```
import { useState } from 'react'
import './App.css'

function HandicraftCard({ itemName, origin, price }) {
  const [count, setCount] = useState(0)

  const handleIncrement = () => {
    setCount(count + 1)
  }

  const handleDecrement = () => {
    if (count > 0) {
      setCount(count - 1)
    }
  }

  const handleOrder = () => {
    if (count > 0) {
      alert(`Order placed for ${count} piece(s) of ${itemName}!`)
    } else {
      alert('Please select quantity first.')
    }
  }

  return (
    <div className="product-card">
      <h2>{itemName}</h2>
      <p className="origin"><strong>Origin:</strong> {origin}</p>
      <p className="price">₹{price}</p>

      <div className="quantity-section">
        <button onClick={handleDecrement}>−</button>
        <span>{count}</span>
        <button onClick={handleIncrement}>+</button>
      </div>

      <p className="total-text">
        Total: <strong>₹{price * count}</strong>
      </p>
```

```
      <button className="cart-button" onClick={handleOrder}>
        Add to Cart
      </button>
    </div>
  )
}

function App() {
  return (
    <div className="app">
      <h1>Traditional Handicrafts Store</h1>
      <p className="subtitle">
        Props, State Management and Event Handling Demo
      </p>

      <div className="product-container">
        <HandicraftCard
          itemName="Banarasi Silk Saree"
          origin="Varanasi (UP)"
          price={3200}
        />

        <HandicraftCard
          itemName="Dhokra Brass Figurine"
          origin="Bastar (Chhattisgarh)"
          price={1250}
        />

        <HandicraftCard
          itemName="Channapatna Wooden Toys"
          origin="Karnataka"
          price={650}
        />
      </div>
    </div>
  )
}

export default App
```

2. App.css

```
#root {
  width: 100%;
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  background-color: #f8fafc;
  color: #334155;
  min-height: 100vh;
}

.app {
  min-height: 100vh;
  padding: 50px 20px;
  text-align: center;
  max-width: 1100px;
  margin: 0 auto;
}

.app h1 {
  margin-bottom: 10px;
  color: #1e3a8a;
  font-size: 2.2rem;
  font-weight: 700;
}

.subtitle {
  color: #64748b;
  margin-bottom: 35px;
  font-size: 1rem;
}

.product-container {
  display: flex;
  justify-content: center;
  gap: 25px;
  flex-wrap: wrap;
}
```

```
.product-card {
  width: 300px;
  padding: 25px;
  background: #ffffff;
  border-radius: 12px;
  box-shadow: 0 4px 15px rgba(0, 0, 0, 0.06);
  border: 1px solid #e2e8f0;
  border-top: 4px solid #0284c7;
  text-align: left;
  transition: transform 0.2s ease, box-shadow 0.2s ease;
}

.product-card:hover {
  transform: translateY(-4px);
  box-shadow: 0 8px 20px rgba(0, 0, 0, 0.1);
}

.product-card h2 {
  margin-top: 0;
  margin-bottom: 10px;
  font-size: 1.25rem;
  color: #0f172a;
  text-align: center;
}

.origin {
  font-size: 0.9rem;
  color: #64748b;
  margin-bottom: 12px;
  text-align: center;
}

.price {
  font-size: 1.25rem;
  color: #059669;
  font-weight: bold;
  margin-bottom: 15px;
  text-align: center;
}

.quantity-section {
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 15px;
  margin-bottom: 15px;
  background: #f1f5f9;
  padding: 8px;
  border-radius: 8px;
}

.quantity-section button {
  background-color: #0284c7;
  color: white;
  border: none;
  width: 32px;
  height: 32px;
  border-radius: 50%;
  font-size: 16px;
  cursor: pointer;
  transition: background-color 0.2s;
  display: flex;
  align-items: center;
  justify-content: center;
}

.quantity-section button:hover {
  background-color: #0369a1;
}

.quantity-section span {
  font-size: 1.1rem;
  font-weight: bold;
  min-width: 24px;
  text-align: center;
  color: #0f172a;
}

.total-text {
  text-align: center;
  margin-bottom: 15px;
  font-size: 1rem;
  color: #334155;
}

.cart-button {
  width: 100%;
```

```
  padding: 11px;
  background-color: #059669;
  color: white;
  border: none;
  border-radius: 6px;
  font-size: 1rem;
  font-weight: 600;
  cursor: pointer;
  transition: background-color 0.2s;
}

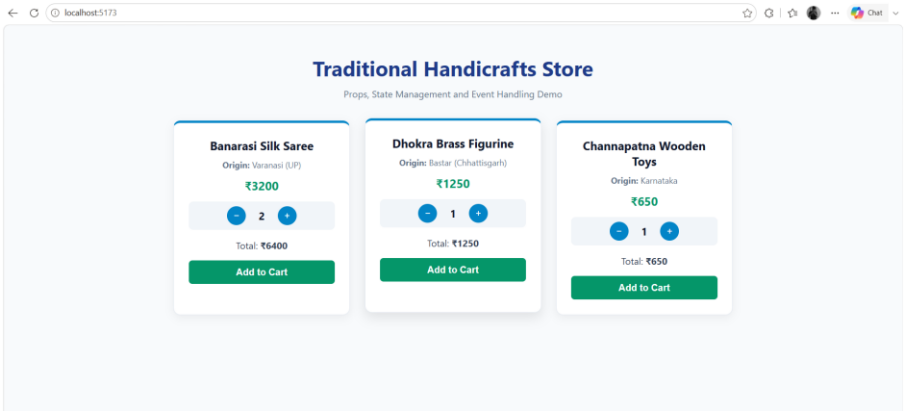
.cart-button:hover {
  background-color: #047857;
}
```

3. main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Output



Result

Thus, the React application was successfully developed using Props, State Management, and Event Handling concepts. The application successfully handled user interactions and dynamically updated the user interface using React state.

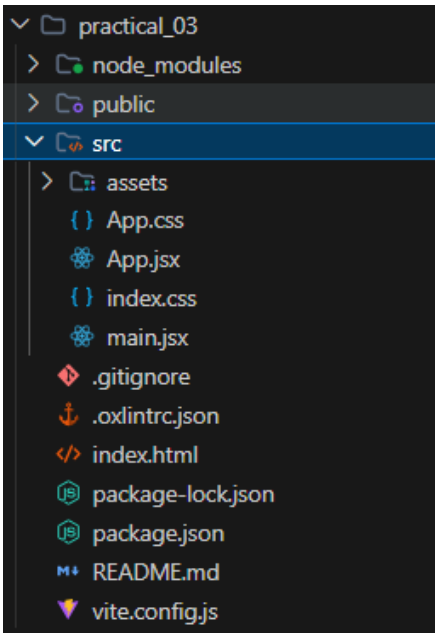
PRACTICAL NO. 03

Create a Student Management System using React Forms, Controlled Components and Validation.

Software Requirements

- Code Editor: Visual Studio Code
- Runtime Environment: Node.js
- Package Manager: npm
- JavaScript Library: React.js
- Build Tool: Vite

Folder Structure



Source Code

1. App.jsx

```
import { useState } from 'react'
import './App.css'

function App() {
  const [student, setStudent] = useState({
    name: '',
    email: '',
    course: '',
    age: '',
  })

  const [students, setStudents] = useState([])
  const [errors, setErrors] = useState({})

  const handleChange = (event) => {
    const { name, value } = event.target
    setStudent({
      ...student,
      [name]: value,
    })
  }

  const validateForm = () => {
    const newErrors = {}

    if (student.name.trim() === '') {
      newErrors.name = 'Student name is required.'
    }

    if (student.email.trim() === '') {
      newErrors.email = 'Email is required.'
    } else if (!student.email.includes('@')) {
      newErrors.email = 'Enter a valid email address.'
    }

    if (student.course === '') {
      newErrors.course = 'Please select a course.'
    }
```

```
  if (student.age === '') {
    newErrors.age = 'Age is required.'
  } else if (student.age < 17 || student.age > 40) {
    newErrors.age = 'Age must be between 17 and 40.'
  }

  setErrors(newErrors)
  return Object.keys(newErrors).length === 0
}

const handleSubmit = (event) => {
  event.preventDefault()

  if (validateForm()) {
    const newStudent = {
      id: Date.now(),
      ...student,
    }

    setStudents([...students, newStudent])
    setStudent({
      name: '',
      email: '',
      course: '',
      age: '',
    })
    setErrors({})
    alert('Student registered successfully!')
  }
}

const deleteStudent = (id) => {
  setStudents(students.filter((s) => s.id !== id))
}

return (
  <div className="app">
    <h1>Student Information System</h1>

    <div className="form-container">
      <h2>Register New Student</h2>

      <form onSubmit={handleSubmit}>
        <div className="form-group">
          <label>Student Name</label>
          <input
            type="text"
            name="name"
            value={student.name}
            onChange={handleChange}
            placeholder="Enter full name"
          />
          {errors.name && <p className="error">{errors.name}</p>}
        </div>

        <div className="form-group">
          <label>Email Address</label>
          <input
            type="email"
            name="email"
            value={student.email}
            onChange={handleChange}
            placeholder="Enter email address"
          />
          {errors.email && <p className="error">{errors.email}</p>}
        </div>

        <div className="form-group">
          <label>Course</label>
          <select
            name="course"
            value={student.course}
            onChange={handleChange}
          >
            <option value="">Select Course</option>
            <option value="M.Sc. CA & IT">M.Sc. CA & IT</option>
            <option value="B.Sc. CA & IT">B.Sc. CA & IT</option>
            <option value="BCA">BCA</option>
            <option value="MCA">MCA</option>
          </select>
          {errors.course && <p className="error">{errors.course}</p>}
        </div>

        <div className="form-group">
          <label>Age</label>
          <input
            type="number"
```

```
            name="age"
            value={student.age}
            onChange={handleChange}
            placeholder="Enter student age"
          />
          {errors.age && <p className="error">{errors.age}</p>}
        </div>

        <button type="submit" className="add-button">
          Save Student
        </button>
      </form>
    </div>

    <div className="student-list">
      <h2>Registered Students Record</h2>

      {students.length === 0 ? (
        <p className="empty-message">No student records found.</p>
      ) : (
        <div className="table-container">
          <table>
            <thead>
              <tr>
                <th>Name</th>
                <th>Email</th>
                <th>Course</th>
                <th>Age</th>
                <th>Action</th>
              </tr>
            </thead>
            <tbody>
              {students.map((s) => (
                <tr key={s.id}>
                  <td>{s.name}</td>
                  <td>{s.email}</td>
                  <td>{s.course}</td>
                  <td>{s.age}</td>
                  <td>
                    <button
                      className="delete-button"
                      onClick={() => deleteStudent(s.id)}
                    >
                      Delete
                    </button>
                  </td>
                </tr>
              ))}
            </tbody>
          </table>
        </div>
      )}
    </div>
  </div>
  )
}

export default App
```

2. App.css

```
#root {
  width: 100%;
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  background: #f8fafc;
  color: #334155;
}

.app {
  max-width: 900px;
  margin: 0 auto;
  padding: 40px 20px;
}

.app h1 {
  text-align: center;
  color: #0f766e;
  margin-bottom: 30px;
  font-size: 2.2rem;
```

```
}

.form-container, .student-list {
  background: #ffffff;
  padding: 30px;
  border-radius: 10px;
  border: 1px solid #e2e8f0;
  border-top: 4px solid #0d9488;
  box-shadow: 0 4px 15px rgba(0, 0, 0, 0.05);
  margin-bottom: 30px;
}

.form-container h2, .student-list h2 {
  font-size: 1.35rem;
  color: #1e293b;
  margin-bottom: 20px;
  border-bottom: 2px solid #f1f5f9;
  padding-bottom: 10px;
}

.form-group {
  margin-bottom: 18px;
}

.form-group label {
  display: block;
  font-weight: 600;
  margin-bottom: 8px;
  color: #475569;
  font-size: 0.95rem;
}

.form-group input,
.form-group select {
  width: 100%;
  padding: 10px 12px;
  border: 1px solid #cbd5e1;
  border-radius: 6px;
  font-size: 1rem;
  outline: none;
  background: #fff;
  transition: border-color 0.2s;
}

.form-group input:focus,
.form-group select:focus {
  border-color: #0d9488;
}

.error {
  color: #ef4444;
  font-size: 0.85rem;
  margin-top: 5px;
}

.add-button {
  width: 100%;
  padding: 12px;
  background-color: #0d9488;
  color: white;
  border: none;
  border-radius: 6px;
  font-size: 1rem;
  font-weight: 600;
  cursor: pointer;
  transition: background-color 0.2s;
}

.add-button:hover {
  background-color: #0f766e;
}

.empty-message {
  color: #94a3b8;
  font-style: italic;
  text-align: center;
  padding: 20px 0;
}

.table-container {
  overflow-x: auto;
}

table {
  width: 100%;
  border-collapse: collapse;
  margin-top: 10px;
```

```
  text-align: left;
}

th, td {
  padding: 12px 14px;
  border-bottom: 1px solid #e2e8f0;
  font-size: 0.95rem;
}

th {
  background-color: #f1f5f9;
  color: #1e293b;
  font-weight: 600;
}

tr:hover {
  background-color: #f8fafc;
}

.delete-button {
  background-color: #ef4444;
  color: white;
  border: none;
  padding: 6px 14px;
  border-radius: 4px;
  cursor: pointer;
  font-size: 0.85rem;
  transition: background-color 0.2s;
}

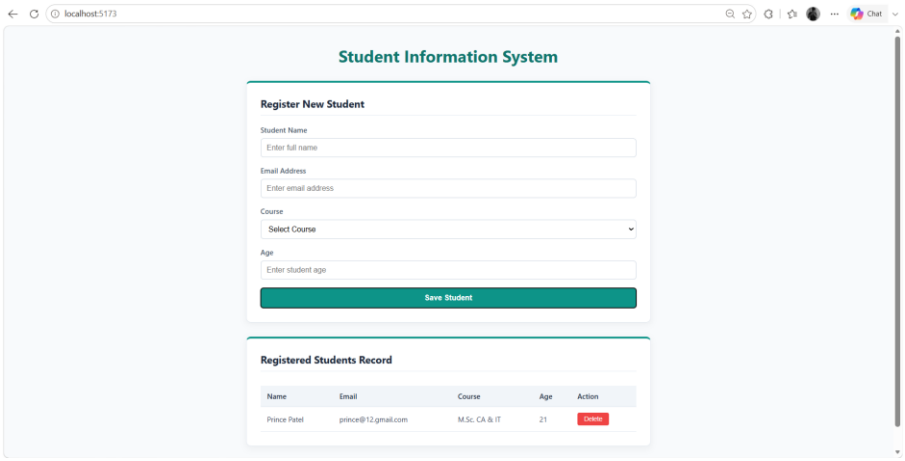
.delete-button:hover {
  background-color: #dc2626;
}
```

3. main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Output



Result

Thus, a Student Management System was successfully developed using React Forms, Controlled Components and Validation. The application successfully accepts student information, validates the user input and manages form data using React state.

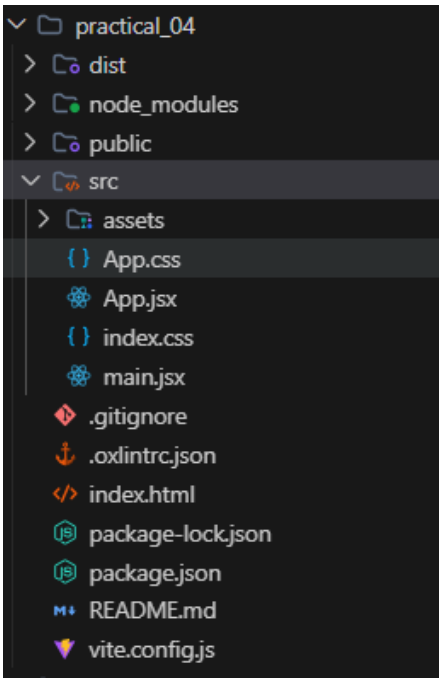
PRACTICAL NO. 04

Develop a Single Page Application (SPA) using React Router for navigation between multiple pages.

Software Requirements

- Code Editor: Visual Studio Code
- Runtime Environment: Node.js
- Package Manager: npm
- JavaScript Library: React.js
- Routing Library: React Router DOM
- Build Tool: Vite
- Web Browser: Google Chrome

Folder Structure



Source Code

1. App.jsx

```
import { BrowserRouter, Routes, Route, NavLink } from 'react-router-dom'
import './App.css'

function Home() {
  return (
    <div className="page">
      <h1>Welcome to Bharat Tourism Portal</h1>
      <p>
        Explore the vibrant heritage, iconic monuments, and diverse cultural landscapes across India.
      </p>
      <p>
        Navigate seamlessly using the menu above to discover famous monuments, prominent regions, and platform details.
      </p>
    </div>
  )
}

function Monuments() {
  const monumentsList = [
    { name: 'Red Fort', state: 'Delhi' },
    { name: 'Meenakshi Temple', state: 'Tamil Nadu' },
    { name: 'Victoria Memorial', state: 'West Bengal' },
  ]

  return (
    <div className="page">
      <h1>Heritage Monuments</h1>

      <div className="card-container">
        {monumentsList.map((item, index) => (
          <div className="card" key={index}>
            <h3>{item.name}</h3>
            <p>State: {item.state}</p>
          </div>
```

```
        ))}
      </div>
    </div>
  )
}

function Regions() {
  return (
    <div className="page">
      <h1>Popular Destinations</h1>

      <div className="card-container">
        <div className="card">
          <h3>Delhi</h3>
          <p>Capital territory showcasing centuries of rich dynastic history and modern monuments.</p>
        </div>

        <div className="card">
          <h3>Tamil Nadu</h3>
          <p>Cradle of Dravidian architecture, grand temple towers, and classical arts.</p>
        </div>

        <div className="card">
          <h3>West Bengal</h3>
          <p>Cultural hub known for colonial architecture, literature, and living traditions.</p>
        </div>
      </div>
    </div>
  )
}

function About() {
  return (
    <div className="page">
      <h1>About This Portal</h1>

      <p>
        This Single Page Application (SPA) is built using React and React Router for seamless navigation without full page reload.
      </p>

      <p>
        It showcases declarative routing, NavLink active state management, and clean modular component architecture.
      </p>
    </div>
  )
}

function NotFound() {
  return (
    <div className="page">
      <h1>404</h1>
      <p>Page Not Found</p>
    </div>
  )
}

function App() {
  return (
    <BrowserRouter>
      <div className="app">
        <nav className="navbar">
          <h2>Bharat Tourism Portal</h2>

          <div className="nav-links">
            <NavLink to="/">Home</NavLink>
            <NavLink to="/monuments">Monuments</NavLink>
            <NavLink to="/regions">Regions</NavLink>
            <NavLink to="/about">About</NavLink>
          </div>
        </nav>

        <main>
          <Routes>
            <Route path="/" element={<Home />} />
            <Route path="/monuments" element={<Monuments />} />
            <Route path="/regions" element={<Regions />} />
            <Route path="/about" element={<About />} />
            <Route path="*" element={<NotFound />} />
          </Routes>
        </main>
      </div>
    </BrowserRouter>
  )
}

export default App
```

2. App.css

```
#root {
  width: 100%;
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  background: #f8fafc;
  color: #334155;
}

.app {
  min-height: 100vh;
  display: flex;
  flex-direction: column;
}

.navbar {
  background-color: #ffffff;
  padding: 18px 40px;
  display: flex;
  justify-content: space-between;
  align-items: center;
  box-shadow: 0 2px 10px rgba(0, 0, 0, 0.05);
  border-bottom: 1px solid #e2e8f0;
}

.navbar h2 {
  color: #0f766e;
  font-size: 1.45rem;
}

.nav-links {
  display: flex;
  gap: 15px;
}

.nav-links a {
  text-decoration: none;
  color: #475569;
  font-weight: 600;
  font-size: 0.95rem;
  padding: 8px 16px;
  border-radius: 6px;
  transition: all 0.2s ease;
}

.nav-links a:hover {
  color: #0d9488;
  background-color: #f0fdfa;
}

.nav-links a.active {
  color: #ffffff;
  background-color: #0d9488;
}

main {
  flex: 1;
  padding: 40px 20px;
  max-width: 1050px;
  margin: 0 auto;
  width: 100%;
}

.page {
  background: #ffffff;
  padding: 40px;
  border-radius: 10px;
  border: 1px solid #e2e8f0;
  box-shadow: 0 4px 15px rgba(0, 0, 0, 0.04);
  text-align: center;
}

.page h1 {
  color: #0f766e;
  margin-bottom: 20px;
  font-size: 2.1rem;
}
```

```
.page p {
  color: #475569;
  font-size: 1.05rem;
  line-height: 1.6;
  margin-bottom: 15px;
}

.card-container {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
  gap: 20px;
  margin-top: 30px;
  text-align: left;
}

.card {
  background: #f8fafc;
  padding: 22px;
  border-radius: 8px;
  border: 1px solid #e2e8f0;
  border-left: 4px solid #0d9488;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.03);
}

.card h3 {
  color: #1e293b;
  margin-bottom: 8px;
  font-size: 1.25rem;
}

.card p {
  font-size: 0.95rem;
  color: #64748b;
  margin: 0;
}
```

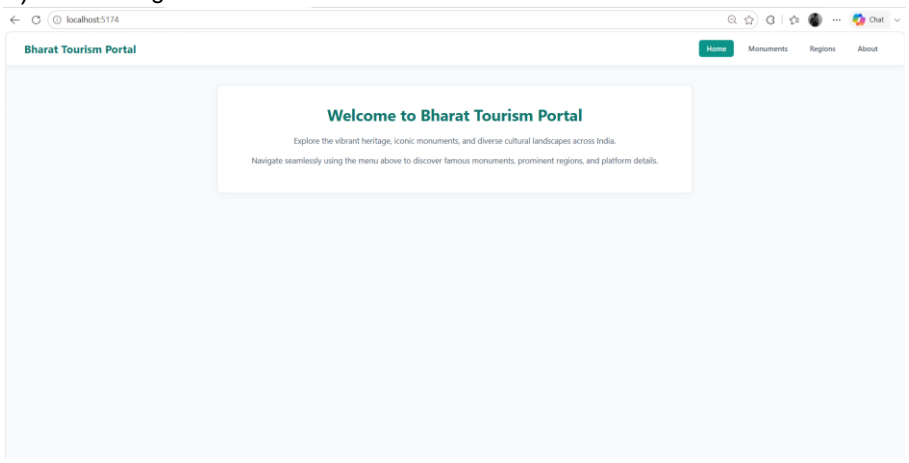
3. main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

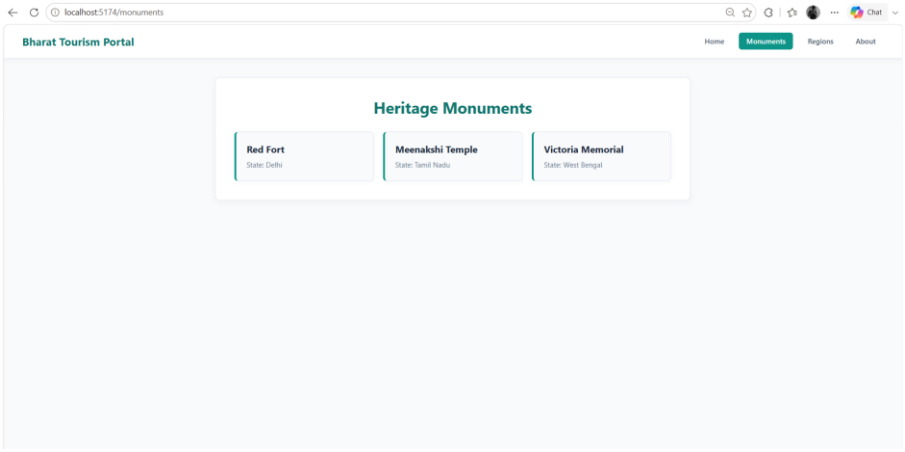
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Output

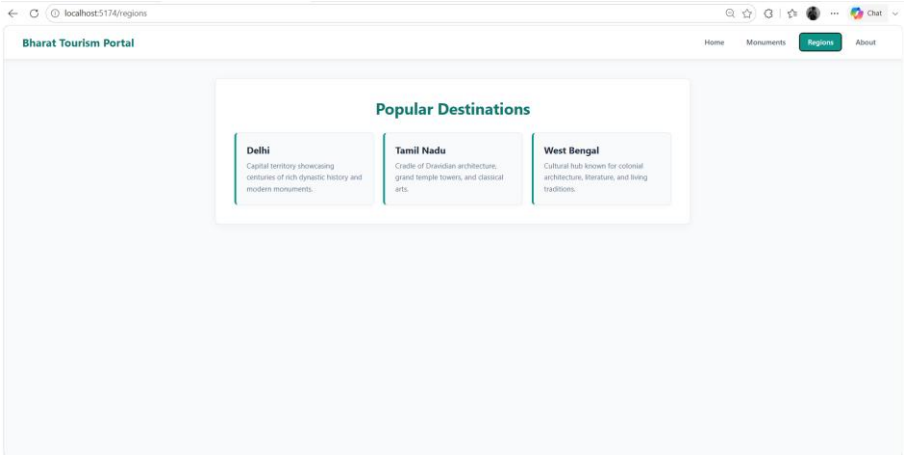
1) Home Page



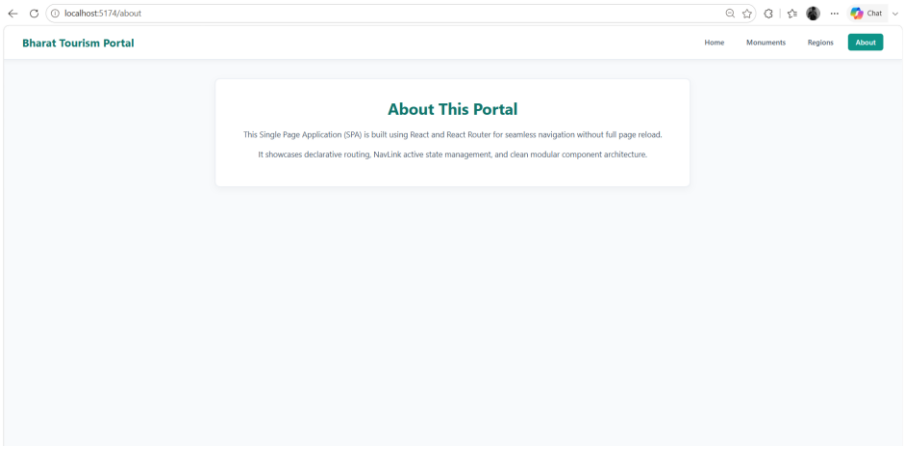
2) Monuments Page



3) Regions Page



4) About Page



Result

Thus, a Single Page Application (SPA) was successfully developed using React Router. The application provides navigation between multiple pages without reloading the complete web page and successfully demonstrates BrowserRouter, Routes, Route and NavLink concepts.

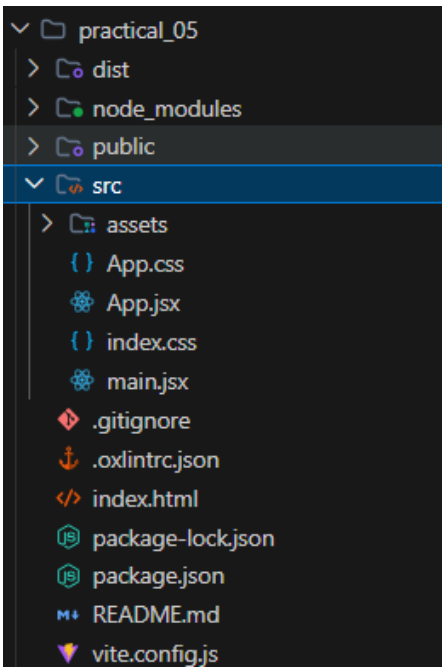
PRACTICAL NO. 05

Develop a React application consuming REST APIs using Axios and Fetch API.

Software Requirements

- Code Editor: Visual Studio Code
- Runtime Environment: Node.js
- Package Manager: npm
- JavaScript Library: React.js
- Build Tool: Vite
- HTTP Client Library: Axios
- REST API: JSONPlaceholder & DummyJSON
- Web Browser: Google Chrome

Folder Structure



Source Code

1. App.jsx

```
import { useState, useEffect } from 'react'
import axios from 'axios'
import './App.css'

function App() {
  const [teamMembers, setTeamMembers] = useState([]) // Using Fetch API
  const [products, setProducts] = useState([]) // Using Axios
  const [loadingMembers, setLoadingMembers] = useState(true)
  const [loadingProducts, setLoadingProducts] = useState(true)

  // 1. Fetch API - Team Members Data
  useEffect(() => {
    fetch('https://jsonplaceholder.typicode.com/users')
      .then((response) => response.json())
      .then((data) => {
        const teamNames = ['Aditya Kulkarni', 'Ananya Deshmukh', 'Vikramaditya Rao', 'Ishita Sengupta']
        const engineeringTeam = data.slice(0, 4).map((user, index) => ({
          id: user.id,
          name: teamNames[index],
          role: ['Cloud Architect', 'Frontend Engineer', 'API Specialist', 'Data Engineer'][index],
          email: user.email,
        }))
        setTeamMembers(engineeringTeam)
        setLoadingMembers(false)
      })
      .catch((error) => console.error('Error with Fetch API:', error))
  }, [])

  // 2. Axios - Featured Products Data
  useEffect(() => {
    axios
      .get('https://dummyjson.com/products?limit=4')
      .then((response) => {
```

```
        setProducts(response.data.products)
        setLoadingProducts(false)
      })
      .catch((error) => console.error('Error with Axios:', error))
  }, [])

  return (
    <div className="app">
      <h1>REST API Integration Dashboard</h1>
      <p className="subtitle">Consuming REST APIs using Fetch API and Axios</p>

      <section className="api-section">
        <h2>Engineering Team (Retrieved via Fetch API)</h2>
        {loadingMembers ? (
          <p className="loading">Loading team members...</p>
        ) : (
          <div className="card-container">
            {teamMembers.map((member) => (
              <div className="card" key={member.id}>
                <h3>{member.name}</h3>
                <p><strong>Role:</strong> {member.role}</p>
                <p><strong>Email:</strong> {member.email}</p>
              </div>
            ))}
          </div>
        )}
      </section>

      <section className="api-section">
        <h2>Featured Products (Retrieved via Axios)</h2>
        {loadingProducts ? (
          <p className="loading">Loading products...</p>
        ) : (
          <div className="card-container">
            {products.map((item) => (
              <div className="card" key={item.id}>
                <h3>{item.title}</h3>
                <p className="price">₹{(item.price * 80).toFixed(0)}</p>
                <p><strong>Category:</strong> {item.category}</p>
              </div>
            ))}
          </div>
        )}
      </section>
    </div>
  )
}

export default App
```

2. App.css

```
#root {
  width: 100%;
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  background: #f8fafc;
  color: #334155;
}

.app {
  max-width: 1100px;
  margin: 0 auto;
  padding: 40px 20px;
}

.app h1 {
  text-align: center;
  color: #0f766e;
  font-size: 2.2rem;
  margin-bottom: 5px;
}

.subtitle {
  text-align: center;
  color: #64748b;
  margin-bottom: 35px;
  font-size: 1rem;
}
```

```
.api-section {
  background: #ffffff;
  padding: 30px;
  border-radius: 10px;
  border: 1px solid #e2e8f0;
  border-top: 4px solid #0d9488;
  box-shadow: 0 4px 15px rgba(0, 0, 0, 0.04);
  margin-bottom: 30px;
}

.api-section h2 {
  font-size: 1.3rem;
  color: #1e293b;
  margin-bottom: 20px;
  border-bottom: 2px solid #f1f5f9;
  padding-bottom: 10px;
}

.card-container {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 20px;
}

.card {
  background: #f8fafc;
  padding: 20px;
  border-radius: 8px;
  border: 1px solid #e2e8f0;
  border-left: 4px solid #0d9488;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.03);
}

.card h3 {
  font-size: 1.1rem;
  color: #1e293b;
  margin-bottom: 10px;
}

.card p {
  font-size: 0.9rem;
  color: #475569;
  margin-bottom: 6px;
}

.price {
  color: #059669 !important;
  font-weight: bold;
  font-size: 1.05rem;
}

.loading {
  text-align: center;
  color: #94a3b8;
  font-style: italic;
  padding: 20px 0;
}
```

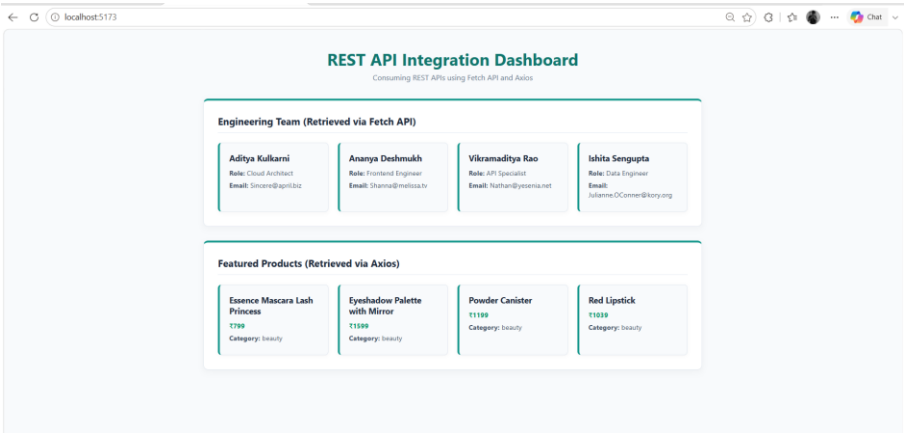
3. main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Output

Final Output Screenshot:
1) Fetch API & Axios section



Result

Thus, a React application was successfully developed to consume REST APIs using Fetch API and Axios. The application successfully retrieved and displayed user data from a REST API while demonstrating state management, side effects, loading states and error handling.

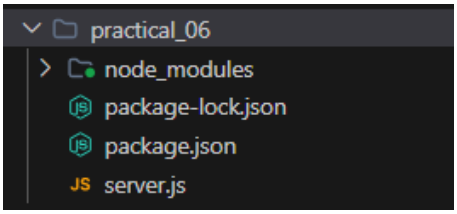
PRACTICAL NO. 06

Create a backend server using Express.js implementing routing, middleware and request handling.

Software Requirements

- Code Editor: Visual Studio Code
- Runtime Environment: Node.js
- Package Manager: npm
- Backend Framework: Express.js
- API Testing Tool: Google Chrome / Postman
- Web Browser: Google Chrome

Folder Structure



Source Code

1. server.js

```
const express = require('express')

const app = express()

const PORT = 5000


app.use(express.json())


const requestLogger = (req, res, next) => {
  console.log(`${req.method} ${req.url}`)
  next()
}

app.use(requestLogger)


app.get('/', (req, res) => {
  res.json({
    message: 'Welcome to Student Management API'
  })
})


app.get('/api/students', (req, res) => {
  const students = [
    {
      id: 1,
      name: 'Aryan Joshi',
      course: 'M.Sc. CA & IT'
    },
    {
      id: 2,
      name: 'Divya Mehta',
      course: 'M.Sc. CA & IT'
    },
    {
      id: 3,
      name: 'Karan Sharma',
      course: 'MCA'
    }
  ]

  res.json(students)
})


app.get('/api/students/:id', (req, res) => {
  const studentId = req.params.id

  res.json({
    message: 'Student requested successfully',
    studentId: studentId
  })
})
```

```
app.post('/api/students', (req, res) => {
  const student = req.body

  res.status(201).json({
    message: 'Student added successfully',
    student: student
  })
})


app.use((req, res) => {
  res.status(404).json({
    message: 'Route not found'
  })
})


app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`)
})
```

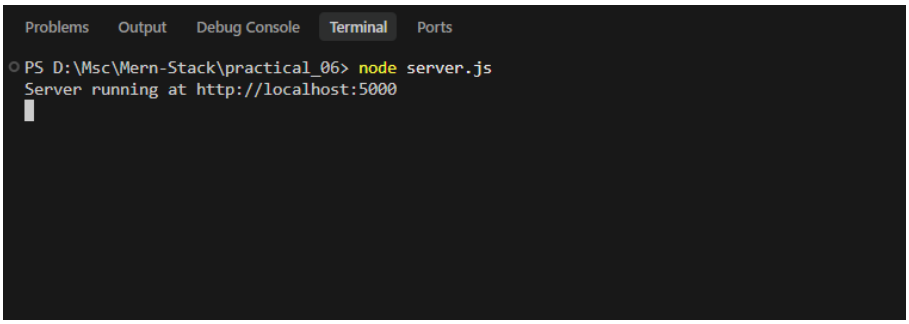
2. package.json

```
{
  "name": "practical_06",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "express": "^5.2.1"
  }
}
```

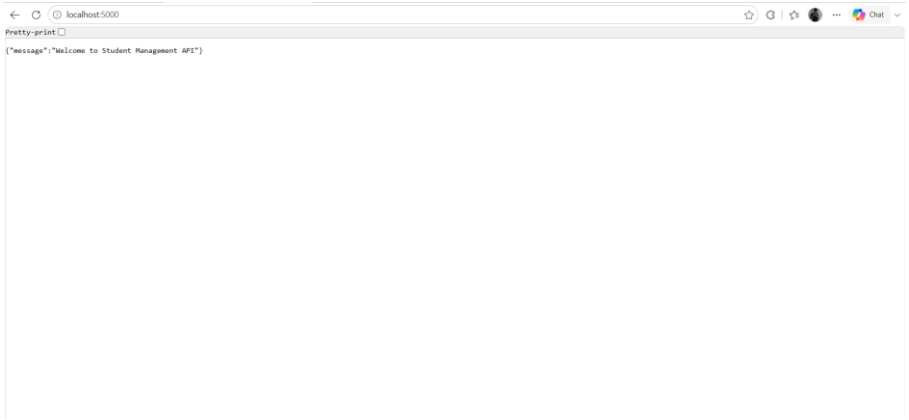
Output

Final Output Screenshot:

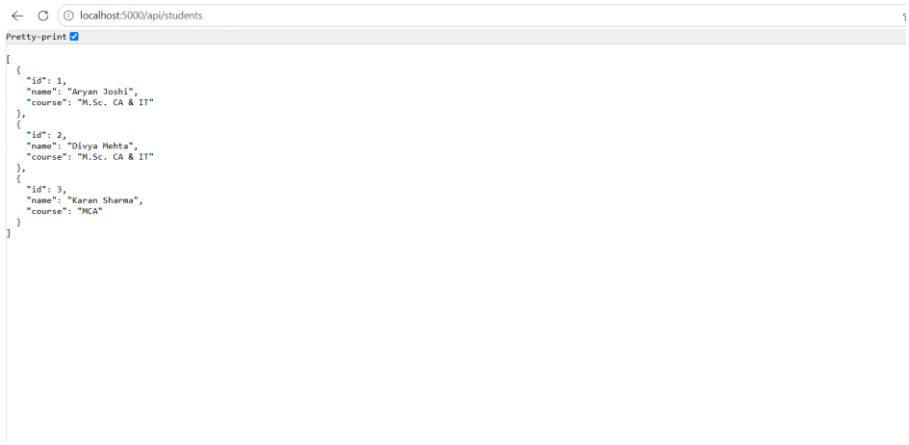
1) Terminal



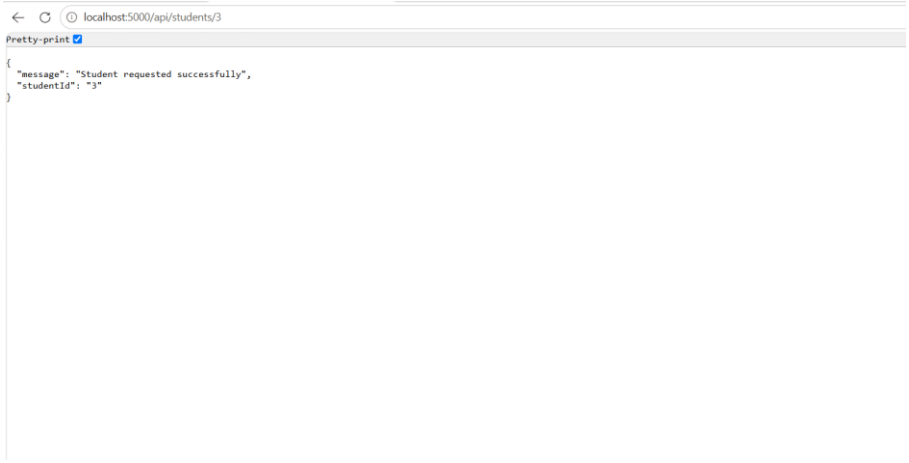
2) Browser Home Route



3) Browser Students Route



4) Browser Single Student Route



Result

Thus, a backend server was successfully created using Express.js. The application successfully demonstrated routing, middleware and request handling by processing GET and POST requests and sending JSON responses to the client.

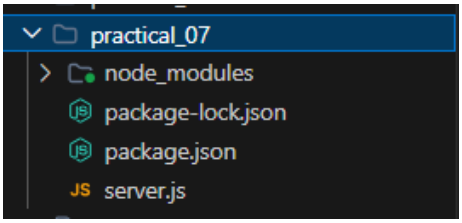
PRACTICAL NO. 07

Build RESTful APIs for Student Records Management using Express.js and Postman testing.

Software Requirements

- Visual Studio Code
- Node.js
- npm (Node Package Manager)
- Express.js
- Postman
- Google Chrome

Folder Structure



Source Code

1. server.js

```
const express = require('express')

const app = express()

const PORT = 5000


app.use(express.json())


let students = [
  {
    id: 1,
    name: 'Aryan Joshi',
    email: 'aryan@gmail.com',
    course: 'M.Sc. CA & IT'
  },
  {
    id: 2,
    name: 'Divya Mehta',
    email: 'divya@gmail.com',
    course: 'M.Sc. CA & IT'
  },
  {
    id: 3,
    name: 'Karan Sharma',
    email: 'karan@gmail.com',
    course: 'MCA'
  }
]


app.get('/', (req, res) => {
  res.json({
    message: 'Student Records REST API is running'
  })
})


app.get('/api/students', (req, res) => {
  res.json(students)
})


app.get('/api/students/:id', (req, res) => {
  const id = parseInt(req.params.id)

  const student = students.find(
    (student) => student.id === id
  )

  if (!student) {
    return res.status(404).json({
      message: 'Student not found'
    })
```

```
  }

  res.json(student)
})


app.post('/api/students', (req, res) => {
  const { name, email, course } = req.body

  if (!name || !email || !course) {
    return res.status(400).json({
      message: 'Name, email and course are required'
    })
  }

  const newStudent = {
    id: students.length + 1,
    name,
    email,
    course
  }

  students.push(newStudent)

  res.status(201).json({
    message: 'Student added successfully',
    student: newStudent
  })
})


app.put('/api/students/:id', (req, res) => {
  const id = parseInt(req.params.id)

  const student = students.find(
    (student) => student.id === id
  )

  if (!student) {
    return res.status(404).json({
      message: 'Student not found'
    })
  }

  const { name, email, course } = req.body

  if (name) student.name = name
  if (email) student.email = email
  if (course) student.course = course

  res.json({
    message: 'Student updated successfully',
    student
  })
})


app.delete('/api/students/:id', (req, res) => {
  const id = parseInt(req.params.id)

  const studentIndex = students.findIndex(
    (student) => student.id === id
  )

  if (studentIndex === -1) {
    return res.status(404).json({
      message: 'Student not found'
    })
  }

  const deletedStudent = students.splice(
    studentIndex,
    1
  )

  res.json({
    message: 'Student deleted successfully',
    student: deletedStudent[0]
  })
})


app.use((req, res) => {
  res.status(404).json({
    message: 'Route not found'
  })
})
```

```
app.listen(PORT, () => {
  console.log(
    `Server running at http://localhost:${PORT}`
  )
})
```

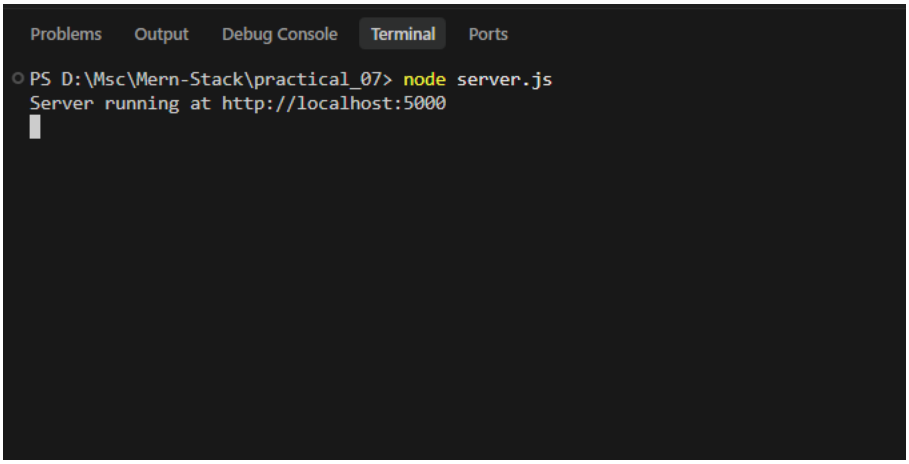
2. package.json

```
{
  "name": "practical_07",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "express": "^5.2.1"
  }
}
```

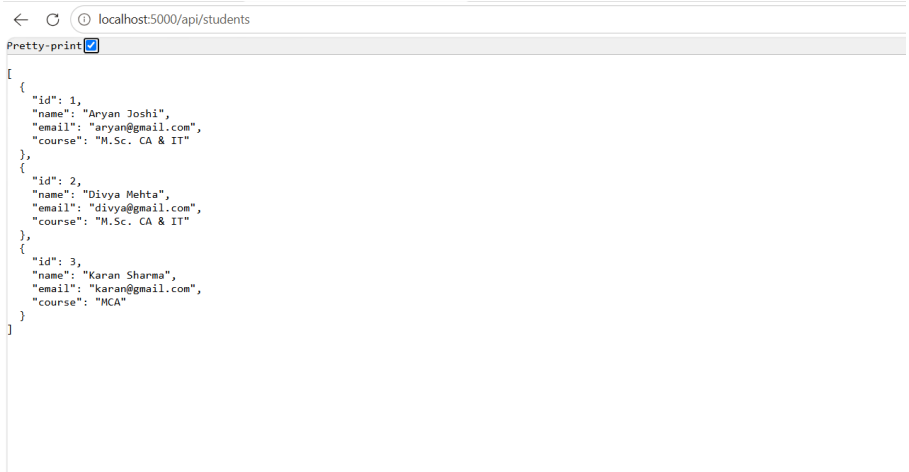
Output

Final Output Screenshot:

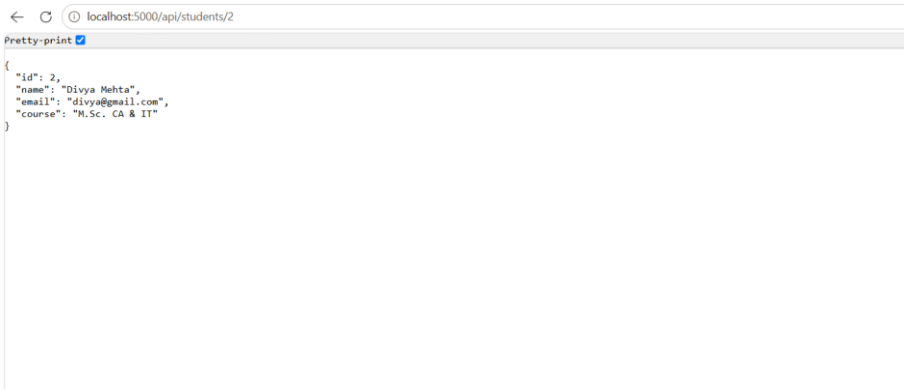
1) Terminal Screenshot



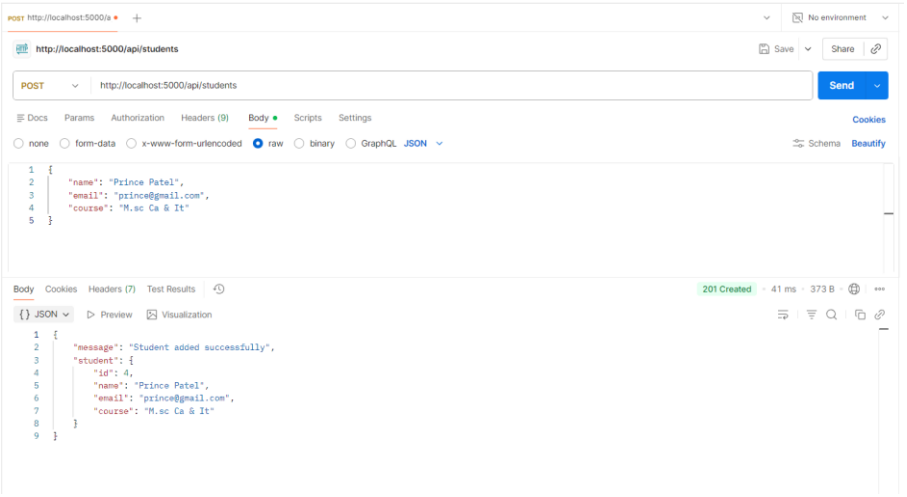
2) GET All Students



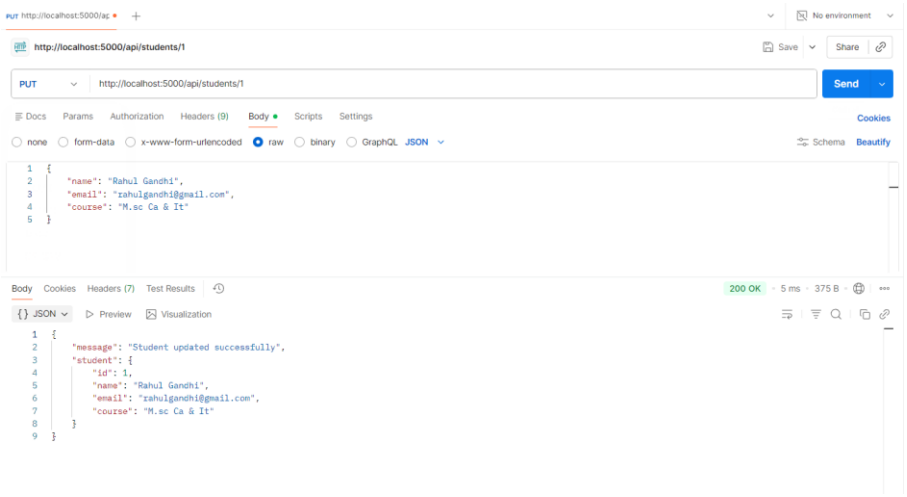
3) GET Student by ID



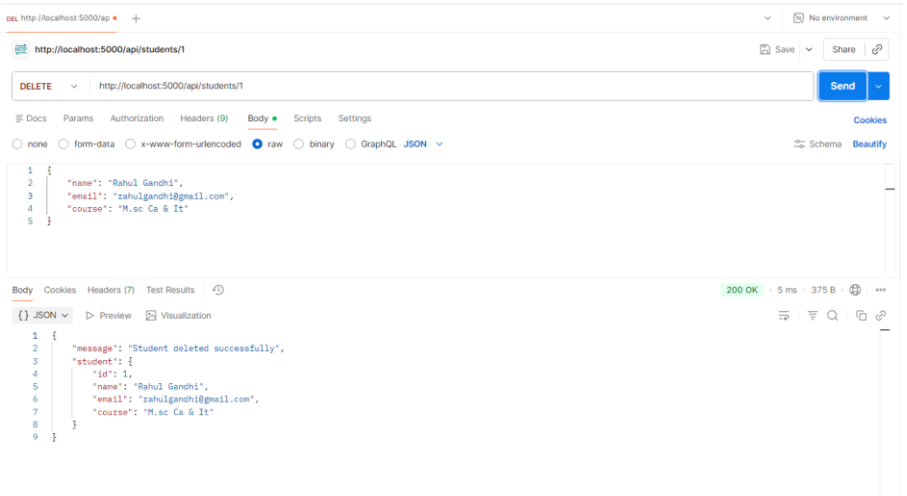
4) POST Student



5) PUT Student



6) DELETE Student



Result

Thus, RESTful APIs for Student Records Management were successfully developed using Express.js and tested using Postman. The application successfully performed CRUD operations such as retrieving, adding, updating and deleting student records.

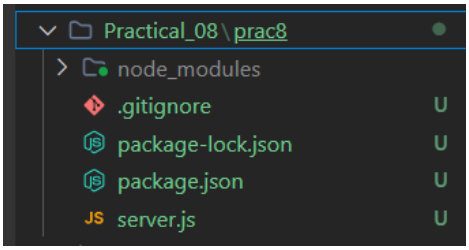
PRACTICAL NO. 08

Create a backend server using Express.js implementing routing, middleware and request handling.

Software Requirements

- Visual Studio Code
- Node.js
- npm (Node Package Manager)
- Express.js
- Postman
- Google Chrome

Folder Structure



Source Code :

1.Server.js :

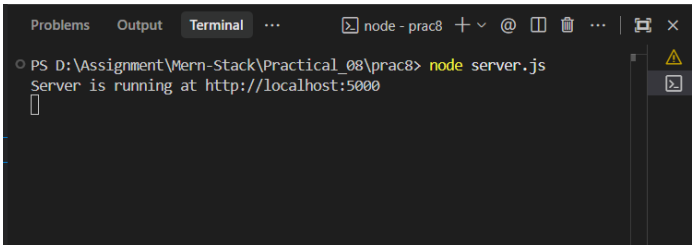
```
const express = require('express')
const app = express()
const PORT = 5000
// In-Memory Data
let users = [
  { id: 1, name: 'Aarav Sharma', role: 'admin' },
  { id: 2, name: 'Diya Patel', role: 'user' },
  { id: 3, name: 'Rohan Verma', role: 'user' }
]
// ==================== 1. MIDDLEWARE ====================
// Built-in Middleware (Parses incoming JSON body)
app.use(express.json())
// Custom Application-level Middleware (Logs method, URL, and time)
const requestLogger = (req, res, next) => {
  console.log(`[${new Date().toLocaleTimeString()}] ${req.method} ${req.url}`)
  next()
}
app.use(requestLogger)
// Route-level Middleware (Validates POST request body)
const validateUser = (req, res, next) => {
  const { name, role } = req.body
  if (!name || !role) {
    return res.status(400).json({ error: 'Name and role are required' })
  }
  next()
}
// ==================== 2. ROUTING & REQUEST HANDLING ====================
// Basic Route
app.get('/', (req, res) => {
  res.json({ message: 'Express Server Running (Practical 08)' })
})
// GET with Query Parameter handling (e.g. /api/users?role=admin)
app.get('/api/users', (req, res) => {
  const { role } = req.query
  if (role) {
    const filteredUsers = users.filter(
      (u) => u.role.toLowerCase() === role.toLowerCase()
    )
```

```
    return res.json(filteredUsers)
  }
  res.json(users)
})
// GET with Route Parameter handling (req.params.id)
app.get('/api/users/:id', (req, res) => {
  const userId = parseInt(req.params.id)
  const user = users.find((u) => u.id === userId)
  if (!user) {
    return res.status(404).json({ error: 'User not found' })
  }
  res.json(user)
})
// POST with Request Body handling (req.body) and validateUser middleware
app.post('/api/users', validateUser, (req, res) => {
  const { name, role } = req.body
  const newUser = { id: users.length + 1, name, role }
  users.push(newUser)
  res.status(201).json({
    message: 'User created successfully',
    user: newUser
  })
})
// 404 Fallback Middleware (Handles undefined routes)
app.use((req, res) => {
  res.status(404).json({ error: 'Route not found' })
})
// ==================== 3. SERVER LISTEN ====================
app.listen(PORT, () => {
  console.log(`Server is running at http://localhost:${PORT}`)
})
```

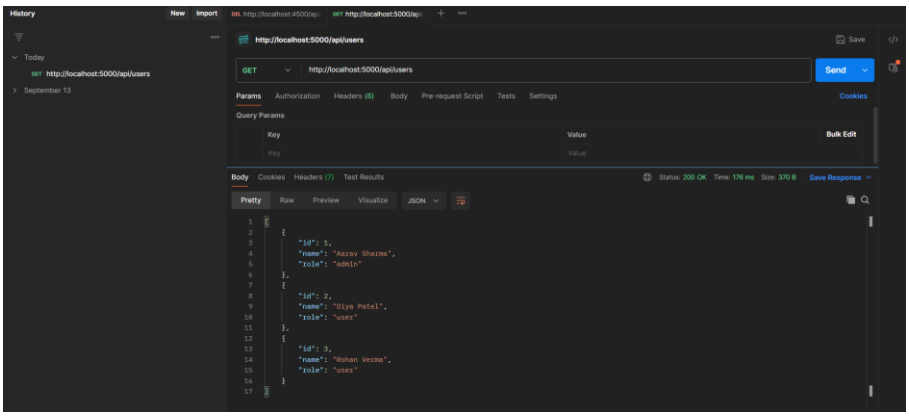
Output

Final Output Screenshot:

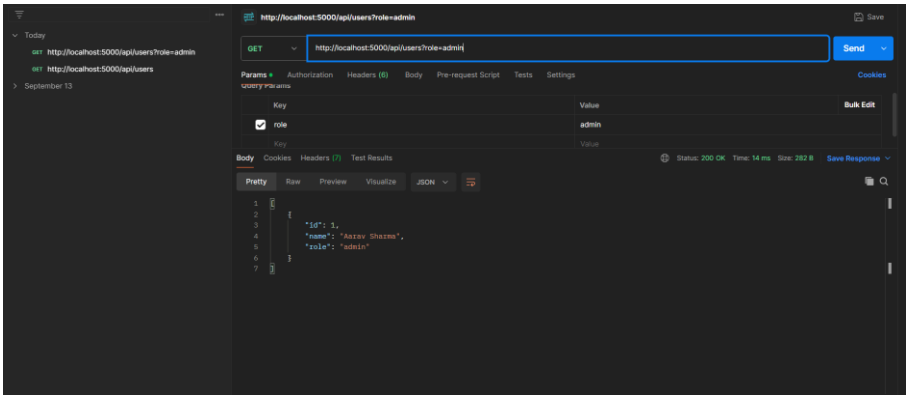
1. Terminal Screenshot



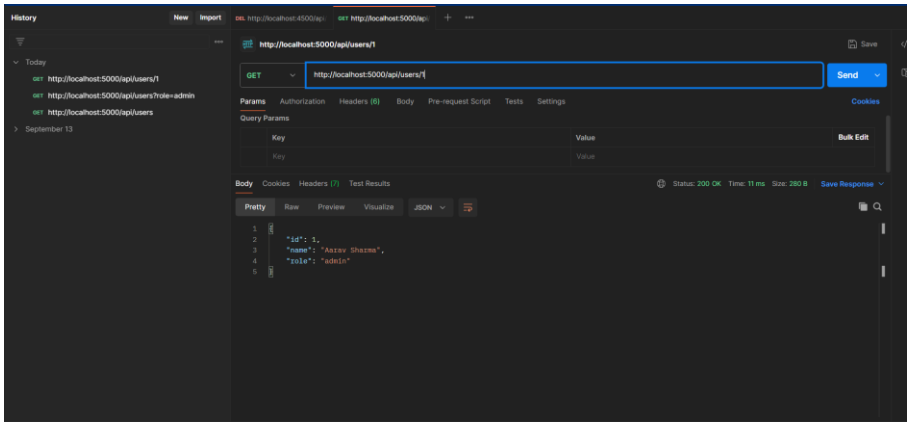
2. GET All Students :



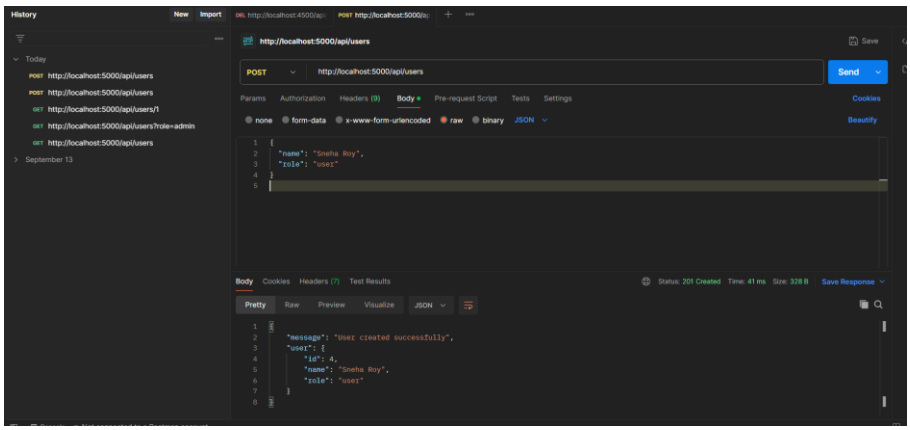
3. **Filter Users by Role :**



4. Get single User by Id :



5. Add New User :



Result :

Thus, a backend server was successfully developed using Express.js implementing routing, middleware, and request handling, and tested using Postman. The application successfully processed HTTP requests using route parameters, query parameters, request body data, and effectively executed built-in, application-level logging, and route-level validation middlewares.

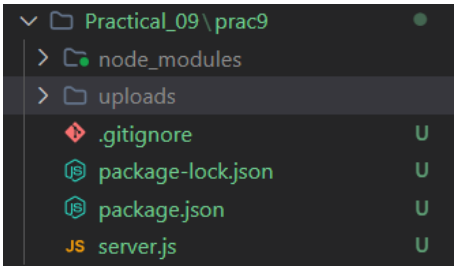
PRACTICAL NO. 09

Create a file upload module using Multer and store file metadata in MongoDB.

1. Software Requirements:

Visual Studio Code
Node.js & npm
Express.js (Backend framework)
Multer (npm i multer — Multipart form-data / file handling)
Mongoose / MongoDB (npm i mongoose — Database connection)
MongoDB
Postman

Folder Structure :



Source Code

1)server.js :

```
const express = require('express')
const mongoose = require('mongoose')
const multer = require('multer')
const path = require('path')
const fs = require('fs')
const app = express()
const PORT = process.env.PORT || 5001
// Middleware to parse JSON
app.use(express.json())
// Serve uploaded files statically (optional preview)
app.use('/uploads', express.static(path.join(__dirname, 'uploads')))
// ==================== 1. MONGODB CONNECTION ====================
const MONGO_URI = 'mongodb://127.0.0.1:27017/fileUploadDB'
mongoose
  .connect(MONGO_URI)
  .then(() => console.log('MongoDB Connected Successfully'))
  .catch((err) => console.error('MongoDB Connection Error:', err.message))
// ==================== 2. MONGOOSE SCHEMA & MODEL ====================
const fileSchema = new mongoose.Schema({
  originalName: { type: String, required: true },
  fileName: { type: String, required: true },
  filePath: { type: String, required: true },
  fileType: { type: String, required: true },
  fileSize: { type: Number, required: true },
  uploadedAt: { type: Date, default: Date.now }
})
const FileMetadata = mongoose.model('FileMetadata', fileSchema)
// ==================== 3. MULTER CONFIGURATION ====================
const uploadDir = path.join(__dirname, 'uploads')
if (!fs.existsSync(uploadDir)) {
  fs.mkdirSync(uploadDir, { recursive: true })
}
const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, uploadDir)
  },
  filename: (req, file, cb) => {
    const uniqueName = `${Date.now()}-${file.originalname}`
```

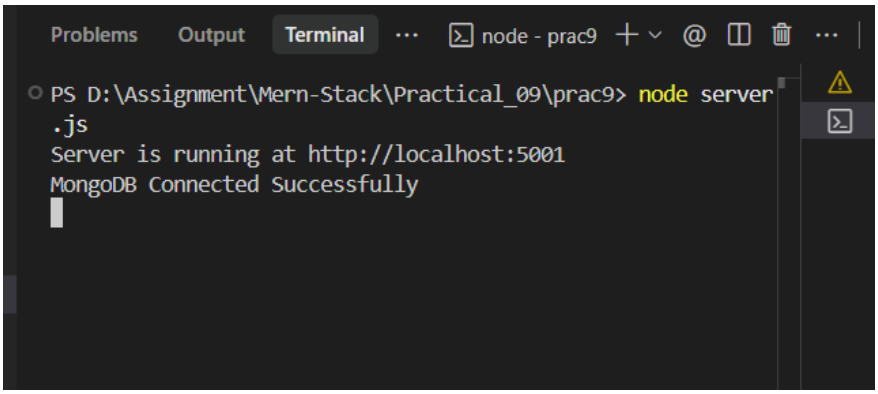
```
    cb(null, uniqueName)
  }
})
const upload = multer({
  storage: storage,
  limits: { fileSize: 10 * 1024 * 1024 } // 10MB limit
})
// ==================== 4. ROUTES ====================
// Root Route
app.get('/', (req, res) => {
  res.json({ message: 'Multer & MongoDB File Upload API is running' })
})
// POST: Upload single file and save metadata in MongoDB
app.post('/api/upload', upload.single('file'), async (req, res) => {
  try {
    if (!req.file) {
      return res.status(400).json({ error: 'Please select a file to upload' })
    }
    // Create file metadata document
    const newFile = new FileMetadata({
      originalName: req.file.originalname,
      fileName: req.file.filename,
      filePath: req.file.path,
      fileType: req.file.mimetype,
      fileSize: req.file.size
    })
    const savedFile = await newFile.save()
    res.status(201).json({
      message: 'File uploaded and metadata stored successfully',
      file: savedFile
    })
  } catch (error) {
    res.status(500).json({ error: error.message })
  }
})
// GET: Retrieve all uploaded files metadata
app.get('/api/files', async (req, res) => {
  try {
    const files = await FileMetadata.find().sort({ uploadedAt: -1 })
    res.json(files)
  } catch (error) {
    res.status(500).json({ error: error.message })
  }
})
// GET: Retrieve single file metadata by ID
app.get('/api/files/:id', async (req, res) => {
  try {
    const file = await FileMetadata.findById(req.params.id)
    if (!file) {
      return res.status(404).json({ error: 'File metadata not found' })
    }
    res.json(file)
  } catch (error) {
    res.status(500).json({ error: 'Invalid file ID or file not found' })
  }
})
// 404 Fallback
app.use((req, res) => {
  res.status(404).json({ error: 'Route not found' })
})
// ==================== 5. SERVER LISTEN ====================
```

```
app.listen(PORT, () => {
  console.log(`Server is running at http://localhost:${PORT}`)
})
```

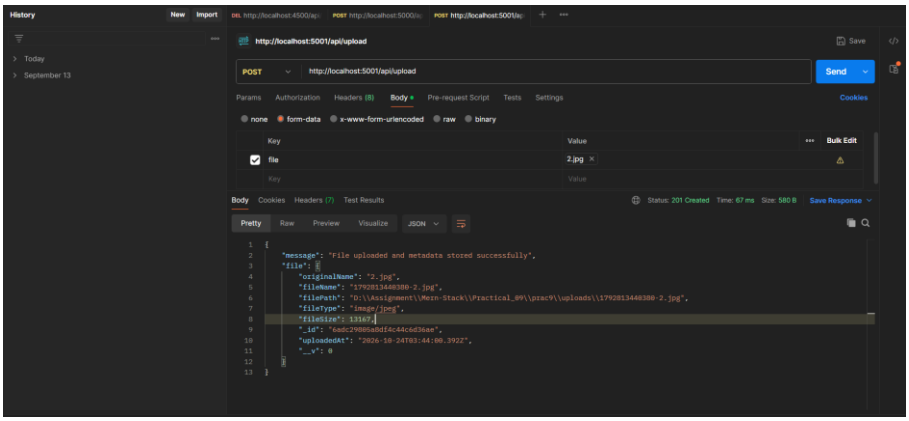
Output

Final Output Screenshot:

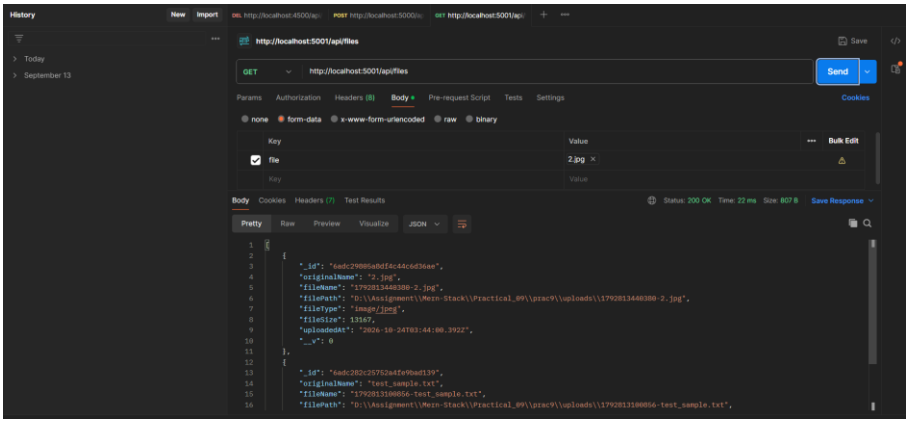
1. Terminal Screenshot



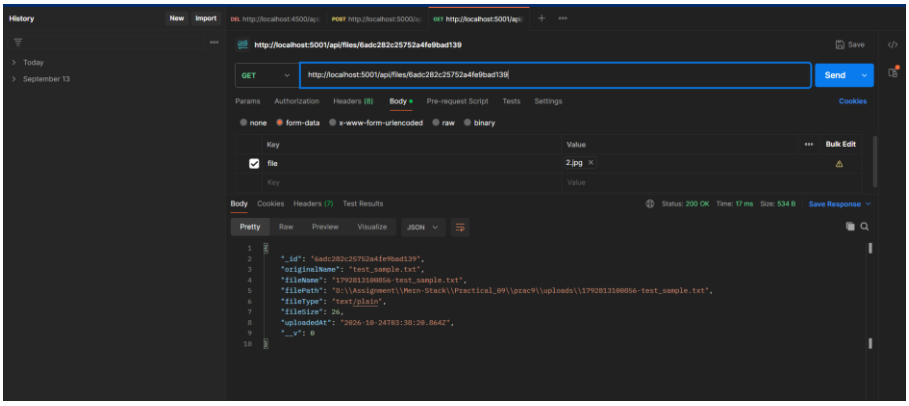
2. Upload File & Store Metadata :



3. Get all files MetaData :



4. Get single File metadata by ID :



Result

Thus, a file upload module was successfully developed using Express.js and Multer middleware, and file metadata was stored in MongoDB using Mongoose. The APIs were tested using Postman with multipart/form-data, successfully saving files to the server's disk storage and their corresponding metadata (file name, type, size, path, and timestamp) in the database.

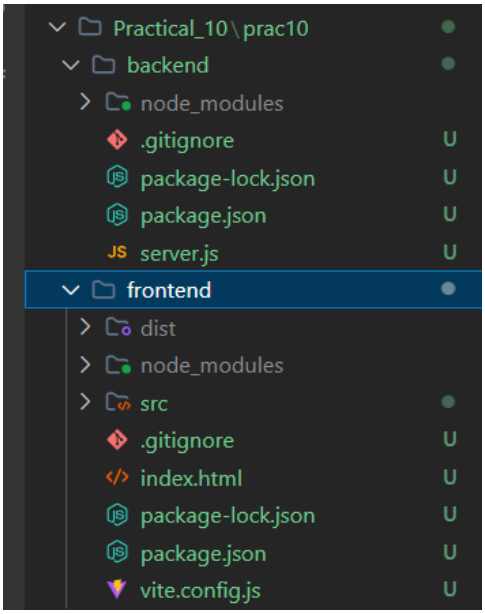
PRACTICAL NO. 10

Develop a complete MERN application integrating React Frontend, Express Backend and MongoDB Database.

2. Software Requirements :

Visual Studio Code
Node.js & npm
MongoDB & MongoDB Compass (Database)
Express.js & Mongoose (Backend)
React.js & Vite (Frontend UI)
CORS (Frontend & Backend connect
Google Chrome

Folder Structure :



Source Code :

BACKEND FOLDER

Server.js :

```
const express = require('express')
const mongoose = require('mongoose')
const cors = require('cors')
const app = express()
const PORT = process.env.PORT || 5002
// Middleware
app.use(cors())
app.use(express.json())
// ==================== 1. MONGODB CONNECTION ====================
const MONGO_URI = 'mongodb://127.0.0.1:27017/mernTaskDB'
mongoose
  .connect(MONGO_URI)
  .then(() => console.log('MongoDB Connected to mernTaskDB'))
  .catch((err) => console.error('MongoDB Connection Error:', err.message))
// ==================== 2. MONGOOSE SCHEMA & MODEL ====================
const taskSchema = new mongoose.Schema({
  title: {
    type: String,
    required: true,
    trim: true
  },
  completed: {
    type: Boolean,
    default: false
  },
  createdAt: {
```

```
    type: Date,
    default: Date.now
  }
})
const Task = mongoose.model('Task', taskSchema)
// ==================== 3. REST API ROUTES ====================
// Root Route
app.get('/', (req, res) => {
  res.json({ message: 'MERN Task Manager API is running' })
})
// GET: Fetch all tasks
app.get('/api/tasks', async (req, res) => {
  try {
    const tasks = await Task.find().sort({ createdAt: -1 })
    res.json(tasks)
  } catch (error) {
    res.status(500).json({ error: error.message })
  }
})
// POST: Create a new task
app.post('/api/tasks', async (req, res) => {
  try {
    const { title } = req.body
    if (!title || !title.trim()) {
      return res.status(400).json({ error: 'Task title is required' })
    }
    const newTask = new Task({ title: title.trim() })
    const savedTask = await newTask.save()
    res.status(201).json({
      message: 'Task added successfully',
      task: savedTask
    })
  } catch (error) {
    res.status(500).json({ error: error.message })
  }
})
// DELETE: Remove a task by ID
app.delete('/api/tasks/:id', async (req, res) => {
  try {
    const deletedTask = await Task.findByIdAndDelete(req.params.id)
    if (!deletedTask) {
      return res.status(404).json({ error: 'Task not found' })
    }
    res.json({
      message: 'Task deleted successfully',
      taskId: req.params.id
    })
  } catch (error) {
    res.status(500).json({ error: 'Invalid task ID' })
  }
})
// 404 Fallback
app.use((req, res) => {
  res.status(404).json({ error: 'Route not found' })
})
// ==================== 4. SERVER LISTEN ====================
app.listen(PORT, () => {
  console.log(`Server is running at http://localhost:${PORT}`)
})
```

➔ Frontend Folder :

APP.jsx :

```
import React, { useState, useEffect } from 'react'
const API_URL = 'http://localhost:5002/api/tasks'
function App() {
  const [tasks, setTasks] = useState([])
  const [title, setTitle] = useState('')
  const [loading, setLoading] = useState(false)
  const [statusMessage, setStatusMessage] = useState('')
  // 1. Fetch all tasks from Express Backend
  const fetchTasks = async () => {
    try {
      setLoading(true)
      const res = await fetch(API_URL)
      const data = await res.json()
      setTasks(data)
    } catch (err) {
      console.error('Failed to fetch tasks:', err)
      setStatusMessage('Error connecting to backend server.')
    } finally {
      setLoading(false)
    }
  }
  useEffect(() => {
    fetchTasks()
  }, [])
  // 2. Add a new task (POST)
  const handleAddTask = async (e) => {
    e.preventDefault()
    if (!title.trim()) return
    try {
      const res = await fetch(API_URL, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ title })
      })
      if (res.ok) {
        setTitle('')
        fetchTasks()
        setStatusMessage('Task added successfully!')
        setTimeout(() => setStatusMessage(''), 3000)
      }
    } catch (err) {
      console.error('Failed to add task:', err)
      setStatusMessage('Error adding task.')
    }
  }
  // 3. Delete a task (DELETE)
  const handleDeleteTask = async (id) => {
    try {
      const res = await fetch(`${API_URL}/${id}`, {
        method: 'DELETE'
      })
      if (res.ok) {
        setTasks(tasks.filter((task) => task._id !== id))
        setStatusMessage('Task deleted successfully!')
        setTimeout(() => setStatusMessage(''), 3000)
      }
    } catch (err) {
      console.error('Failed to delete task:', err)
      setStatusMessage('Error deleting task.')
    }
```

```
  }
  return (
    <div className="app-container">
      <div className="card">
        <header className="header">
          <h1>MERN Task Manager</h1>
          <p className="subtitle">Practical 10: React + Express + MongoDB</p>
        </header>
        {statusMessage && <div className="alert">{statusMessage}</div>}
        {/* Add Task Form */}
        <form onSubmit={handleAddTask} className="task-form">
          <input
            type="text"
            placeholder="Enter a new task..."
            value={title}
            onChange={(e) => setTitle(e.target.value)}
            className="task-input"
          />
          <button type="submit" className="add-btn">
            Add Task
          </button>
        </form>
        {/* Task List */}
        <div className="task-section">
          <h2>Your Tasks ({tasks.length})</h2>
          {loading ? (
            <p className="loading-text">Loading tasks...</p>
          ) : tasks.length === 0 ? (
            <p className="empty-text">No tasks yet. Add your first task above!</p>
          ) : (
            <ul className="task-list">
              {tasks.map((task) => (
                <li key={task._id} className="task-item">
                  <div className="task-info">
                    <span className="task-title">{task.title}</span>
                    <span className="task-date">
                      {new Date(task.createdAt).toLocaleDateString()}
                    </span>
                  </div>
                  <button
                    onClick={() => handleDeleteTask(task._id)}
                    className="delete-btn"
                    title="Delete task"
                  >
                    Delete
                  </button>
                </li>
              ))}
            </ul>
          )}
        </div>
      </div>
    </div>
  )
}
export default App
```

Index.css :

```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
```

```
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, Helvetica, Arial, sans-serif;
}
body {
  background: linear-gradient(135deg, #f5f7fa 0%, #c3cfe2 100%);
  min-height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
  padding: 20px;
}
.app-container {
  width: 100%;
  max-width: 520px;
}
.card {
  background: #ffffff;
  padding: 30px;
  border-radius: 16px;
  box-shadow: 0 10px 25px rgba(0, 0, 0, 0.08);
}
.header {
  text-align: center;
  margin-bottom: 24px;
}
.header h1 {
  font-size: 26px;
  color: #1e293b;
  margin-bottom: 6px;
}
.subtitle {
  font-size: 14px;
  color: #64748b;
}
.alert {
  background-color: #e0f2fe;
  color: #0369a1;
  padding: 10px 14px;
  border-radius: 8px;
  margin-bottom: 18px;
  font-size: 13px;
  text-align: center;
}
.task-form {
  display: flex;
  gap: 10px;
  margin-bottom: 24px;
}
.task-input {
  flex: 1;
  padding: 12px 16px;
  border: 1px solid #cbd5e1;
  border-radius: 8px;
  font-size: 15px;
  outline: none;
  transition: border-color 0.2s;
}
.task-input:focus {
  border-color: #3b82f6;
  box-shadow: 0 0 0 3px rgba(59, 130, 246, 0.15);
}
.add-btn {
```

```
  background-color: #2563eb;
  color: #ffffff;
  border: none;
  padding: 12px 20px;
  border-radius: 8px;
  font-size: 14px;
  font-weight: 600;
  cursor: pointer;
  transition: background-color 0.2s;
}
.add-btn:hover {
  background-color: #1d4ed8;
}
.task-section h2 {
  font-size: 17px;
  color: #334155;
  margin-bottom: 14px;
  padding-bottom: 8px;
  border-bottom: 1px solid #f1f5f9;
}
.loading-text,
.empty-text {
  text-align: center;
  color: #94a3b8;
  font-size: 14px;
  padding: 20px 0;
}
.task-list {
  list-style: none;
  display: flex;
  flex-direction: column;
  gap: 10px;
}
.task-item {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 12px 16px;
  background-color: #f8fafc;
  border: 1px solid #e2e8f0;
  border-radius: 10px;
  transition: transform 0.15s, box-shadow 0.15s;
}
.task-item:hover {
  transform: translateY(-1px);
  box-shadow: 0 4px 6px rgba(0, 0, 0, 0.03);
}
.task-info {
  display: flex;
  flex-direction: column;
  gap: 4px;
}
.task-title {
  font-size: 15px;
  color: #1e293b;
  font-weight: 500;
}
.task-date {
  font-size: 11px;
  color: #94a3b8;
}
```

```
.delete-btn {
  background-color: #fee2e2;
  color: #ef4444;
  border: none;
  padding: 6px 12px;
  border-radius: 6px;
  font-size: 12px;
  font-weight: 600;
  cursor: pointer;
  transition: background-color 0.2s;
}
.delete-btn:hover {
  background-color: #fecaca;
}
```

Main.jsx :

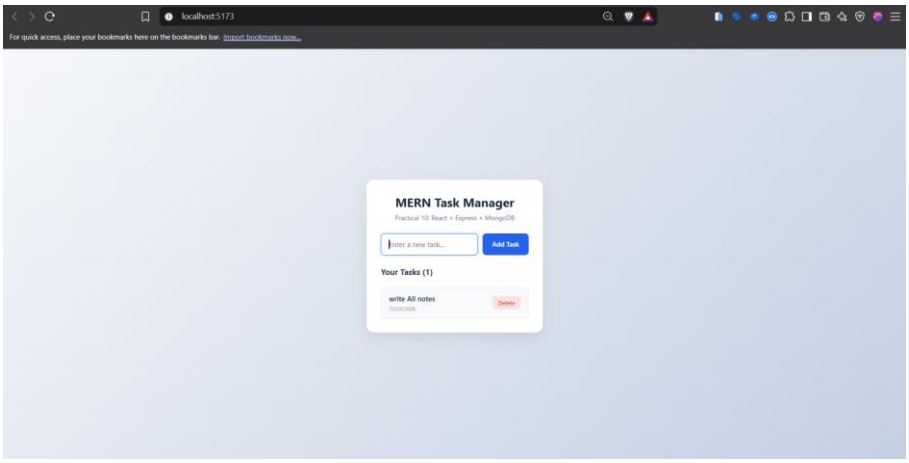
```
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App.jsx'
import './index.css'
ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
)
```

Index.html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>MERN Task Manager - Practical 10</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

OUTPUT :

Screenshot :



Result

Thus, a complete MERN stack application (Task Manager) was successfully developed by integrating a React.js frontend, an Express.js and Node.js backend REST API, and a MongoDB database using Mongoose ODM. The application was tested in the browser and successfully performed full-stack CRUD operations, seamlessly sending asynchronous HTTP requests between client, server, and database.

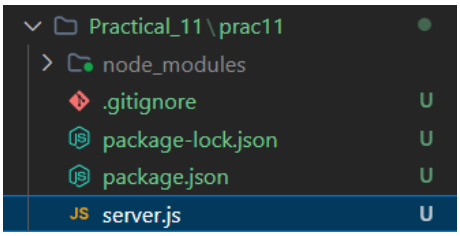
PRACTICAL NO. 11

Create a Blog Management System implementing CRUD Operations, Authentication and Role-Based Access Control.

Software Requirements

Visual Studio Code
Node.js & npm
Express.js (Backend framework)
MongoDB & Mongoose (Database)
JSON Web Token (jsonwebtoken) — Authentication & secure tokens generate
Bcryptjs (bcryptjs) — Password securely hash/encrypt
Postman — Protected routes & JWT Bearer Token test

Folder Structure



Source Code

Server.js :

```
const express = require('express')
const mongoose = require('mongoose')
const bcrypt = require('bcryptjs')
const jwt = require('jsonwebtoken')
const app = express()
const PORT = process.env.PORT || 5003
const JWT_SECRET = 'mern_blog_rbac_secret_key_123'
// Middleware to parse incoming JSON
app.use(express.json())
// ==================== 1. MONGODB CONNECTION ====================
const MONGO_URI = 'mongodb://127.0.0.1:27017/blogRbacDB'
mongoose
  .connect(MONGO_URI)
  .then(() => console.log('MongoDB Connected to blogRbacDB'))
  .catch((err) => console.error('MongoDB Connection Error:', err.message))
// ==================== 2. SCHEMAS & MODELS ====================
// User Schema (with Roles: 'user' or 'admin')
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  role: { type: String, enum: ['user', 'admin'], default: 'user' }
})
const User = mongoose.model('User', userSchema)
// Blog Schema
const blogSchema = new mongoose.Schema({
  title: { type: String, required: true },
  content: { type: String, required: true },
  author: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  authorName: { type: String },
  createdAt: { type: Date, default: Date.now }
})
const Blog = mongoose.model('Blog', blogSchema)
// ==================== 3. AUTH & RBAC MIDDLEWARES ====================
// Middleware: Authenticate JWT Token
const authenticateToken = (req, res, next) => {
  const authHeader = req.headers['authorization']
  const token = authHeader && authHeader.split(' ')[1] // Bearer <TOKEN>
```

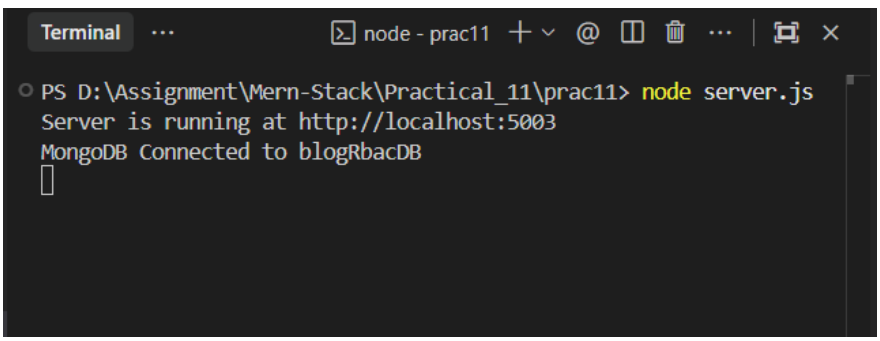
```
  if (!token) {
    return res.status(401).json({ error: 'Access denied: No token provided' })
  }
  jwt.verify(token, JWT_SECRET, (err, decodedUser) => {
    if (err) {
      return res.status(403).json({ error: 'Invalid or expired token' })
    }
    req.user = decodedUser
    next()
  })
}
// Middleware: Role-Based Access Control (RBAC)
const authorizeRole = (requiredRole) => {
  return (req, res, next) => {
    if (req.user.role !== requiredRole) {
      return res.status(403).json({
        error: `Access denied: Requires '${requiredRole}' role permission`
      })
    }
    next()
  }
}
// ==================== 4. AUTHENTICATION ROUTES ====================
// Register New User (role can be 'user' or 'admin')
app.post('/api/auth/register', async (req, res) => {
  try {
    const { name, email, password, role } = req.body
    if (!name || !email || !password) {
      return res.status(400).json({ error: 'All fields are required' })
    }
    const existingUser = await User.findOne({ email })
    if (existingUser) {
      return res.status(400).json({ error: 'Email already registered' })
    }
    // Hash Password
    const salt = await bcrypt.genSalt(10)
    const hashedPassword = await bcrypt.hash(password, salt)
    const newUser = new User({
      name,
      email,
      password: hashedPassword,
      role: role === 'admin' ? 'admin' : 'user'
    })
    await newUser.save()
    res.status(201).json({
      message: 'User registered successfully',
      user: { id: newUser._id, name: newUser.name, email: newUser.email, role: newUser.role }
    })
  } catch (error) {
    res.status(500).json({ error: error.message })
  }
})
// Login User & Generate JWT Token
app.post('/api/auth/login', async (req, res) => {
  try {
    const { email, password } = req.body
    const user = await User.findOne({ email })
    if (!user) {
      return res.status(400).json({ error: 'Invalid email or password' })
    }
```

```
    const isMatch = await bcrypt.compare(password, user.password)
    if (!isMatch) {
      return res.status(400).json({ error: 'Invalid email or password' })
    }
    // Generate JWT Token containing user id and role
    const token = jwt.sign(
      { id: user._id, name: user.name, role: user.role },
      JWT_SECRET,
      { expiresIn: '1d' }
    )
    res.json({
      message: 'Login successful',
      token,
      user: { id: user._id, name: user.name, role: user.role }
    })
  } catch (error) {
    res.status(500).json({ error: error.message })
  }
})
// ==================== 5. BLOG CRUD & RBAC ROUTES ====================
// GET: All Blogs (Public)
app.get('/api/blogs', async (req, res) => {
  try {
    const blogs = await Blog.find().sort({ createdAt: -1 })
    res.json(blogs)
  } catch (error) {
    res.status(500).json({ error: error.message })
  }
})
// GET: Single Blog by ID (Public)
app.get('/api/blogs/:id', async (req, res) => {
  try {
    const blog = await Blog.findById(req.params.id)
    if (!blog) {
      return res.status(404).json({ error: 'Blog not found' })
    }
    res.json(blog)
  } catch (error) {
    res.status(500).json({ error: 'Invalid blog ID' })
  }
})
// POST: Create Blog (Protected: Logged-in User or Admin)
app.post('/api/blogs', authenticateToken, async (req, res) => {
  try {
    const { title, content } = req.body
    if (!title || !content) {
      return res.status(400).json({ error: 'Title and content are required' })
    }
    const newBlog = new Blog({
      title,
      content,
      author: req.user.id,
      authorName: req.user.name
    })
    const savedBlog = await newBlog.save()
    res.status(201).json({
      message: 'Blog created successfully',
      blog: savedBlog
    })
  } catch (error) {
    res.status(500).json({ error: error.message })
```

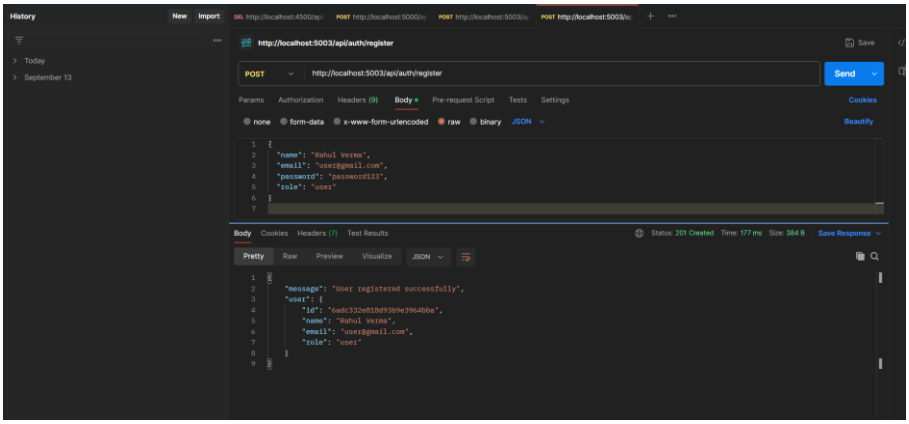
```
  }
})
// PUT: Update Blog (Protected: Only Author can update)
app.put('/api/blogs/:id', authenticateToken, async (req, res) => {
  try {
    const blog = await Blog.findById(req.params.id)
    if (!blog) {
      return res.status(404).json({ error: 'Blog not found' })
    }
    // Check if the logged-in user is the author
    if (blog.author.toString() !== req.user.id && req.user.role !== 'admin') {
      return res.status(403).json({ error: 'You can only edit your own blogs' })
    }
    const { title, content } = req.body
    if (title) blog.title = title
    if (content) blog.content = content
    const updatedBlog = await blog.save()
    res.json({
      message: 'Blog updated successfully',
      blog: updatedBlog
    })
  } catch (error) {
    res.status(500).json({ error: error.message })
  }
})
// DELETE: Delete Blog (RBAC: ONLY Admin can delete)
app.delete('/api/blogs/:id', authenticateToken, authorizeRole('admin'), async (req, res) => {
  try {
    const deletedBlog = await Blog.findByIdAndDelete(req.params.id)
    if (!deletedBlog) {
      return res.status(404).json({ error: 'Blog not found' })
    }
    res.json({
      message: 'Blog deleted successfully by Admin',
      blogId: req.params.id
    })
  } catch (error) {
    res.status(500).json({ error: error.message })
  }
})
// 404 Fallback
app.use((req, res) => {
  res.status(404).json({ error: 'Route not found' })
})
// ==================== 6. SERVER LISTEN ====================
app.listen(PORT, () => {
  console.log(`Server is running at http://localhost:${PORT}`)
})
```

OUTPUT :

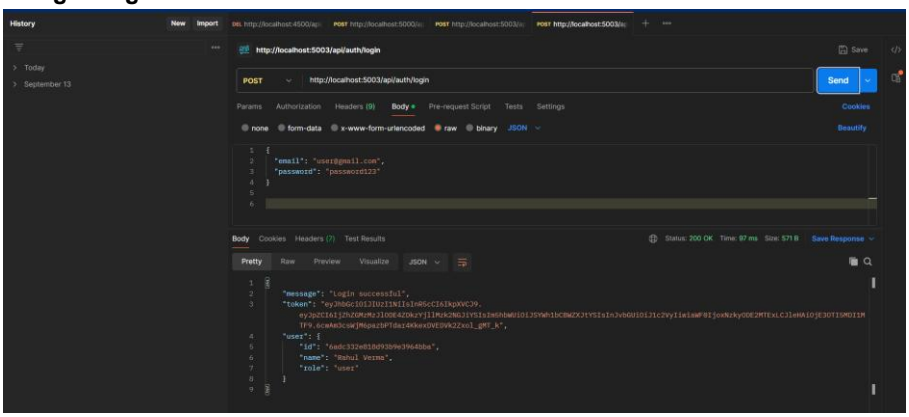
1. Terminal Screenshot :



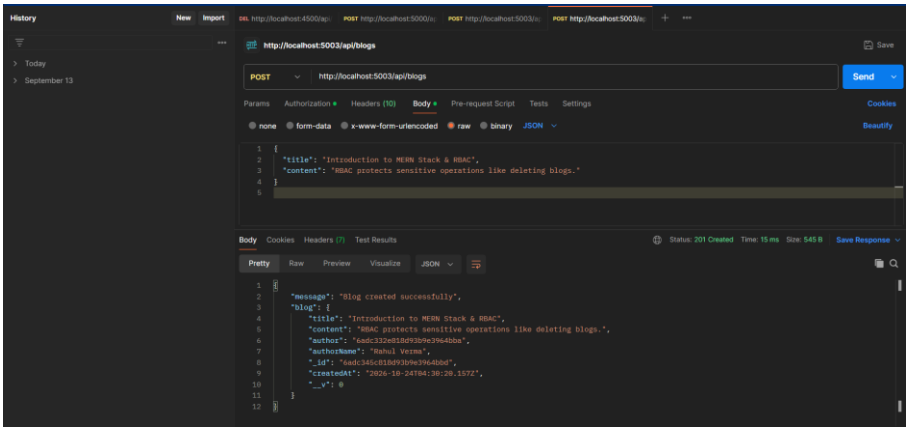
2.Register normal User& Admin :



3.Login to get JWT Token :



4.Create Blog :



Result

Thus, a Blog Management System was successfully developed using Express.js and MongoDB with Mongoose ODM, implementing complete CRUD operations, user authentication with JSON Web Tokens (JWT) and bcrypt password hashing, and Role-Based Access Control (RBAC). The APIs were thoroughly tested using Postman, successfully restricting administrative operations (such as blog deletion) to authorized admin accounts while granting appropriate creation and modification permissions to standard users.

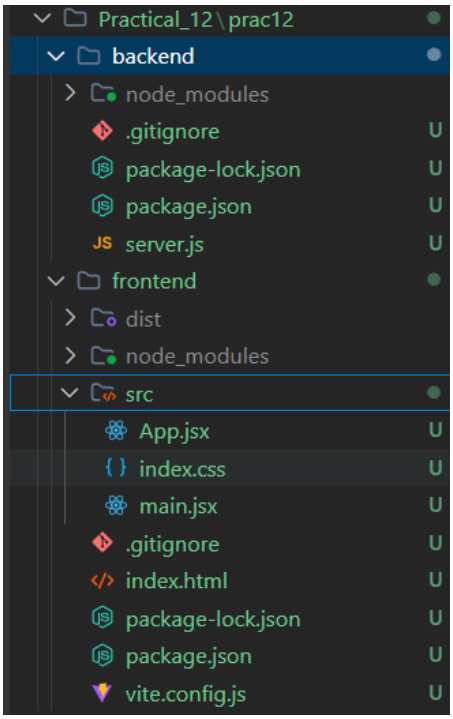
PRACTICAL NO. 12

Develop a Real-Time Chat Application using Socket.IO, React and Node.js.

Software Requirements

Operating System: Windows 10/11
Editor: Visual Studio Code
Runtime Environment: Node.js (v18.x or above)
Package Manager: npm (Node Package Manager)
Backend Technologies: Node.js, Express.js, Socket.IO (socket.io), CORS
Frontend Technologies: React.js, Socket.IO Client (socket.io-client), Vite
Web Browser: Google Chrome (Multiple tabs/windows for live testing)

Folder Structure



Source Code

Backend :

Server.js :

```
const express = require('express')
const http = require('http')
const { Server } = require('socket.io')
const cors = require('cors')
const app = express()
const PORT = process.env.PORT || 5004
// Middleware
app.use(cors())
app.use(express.json())
// Create HTTP Server
const server = http.createServer(app)
// Initialize Socket.IO Server with CORS
const io = new Server(server, {
  cors: {
    origin: '*',
    methods: ['GET', 'POST']
  }
})
// Root Health Check Route
app.get('/', (req, res) => {
  res.json({ message: 'Socket.IO Real-Time Chat Server is running' })
})
// Socket.IO Connection Event
```

```
io.on('connection', (socket) => {
  console.log(`User connected: ${socket.id}`)
  // Listen for 'send_message' from client
  socket.on('send_message', (data) => {
    // Broadcast received message to all connected clients
    io.emit('receive_message', data)
  })
  // Listen for user disconnect
  socket.on('disconnect', () => {
    console.log(`User disconnected: ${socket.id}`)
  })
})
// Start Server
server.listen(PORT, () => {
  console.log(`Chat Server running at http://localhost:${PORT}`)
})
```

➔ Frontend :

App.jsx :

```
import React, { useState, useEffect, useRef } from 'react'
import { io } from 'socket.io-client'
const SOCKET_SERVER_URL = 'http://localhost:5004'
function App() {
  const [socket, setSocket] = useState(null)
  const [username, setUsername] = useState('')
  const [joined, setJoined] = useState(false)
  const [message, setMessage] = useState('')
  const [messages, setMessages] = useState([])
  const messagesEndRef = useRef(null)
  // Auto-scroll to latest message
  const scrollToBottom = () => {
    messagesEndRef.current?.scrollIntoView({ behavior: 'smooth' })
  }
  useEffect(() => {
    scrollToBottom()
  }, [messages])
  // Initialize Socket Connection when user joins
  useEffect(() => {
    if (!joined) return
    const newSocket = io(SOCKET_SERVER_URL)
    setSocket(newSocket)
    // Listen for incoming messages
    newSocket.on('receive_message', (data) => {
      setMessages((prev) => [...prev, data])
    })
    return () => {
      newSocket.disconnect()
    }
  }, [joined])
  // Handle Join
  const handleJoin = (e) => {
    e.preventDefault()
    if (username.trim()) {
      setJoined(true)
    }
  }
  // Handle Send Message
  const handleSendMessage = (e) => {
    e.preventDefault()
    if (!message.trim() || !socket) return
```

```
    const messageData = {
      id: Date.now(),
      sender: username,
      text: message.trim(),
      time: new Date().toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' })
    }
    // Emit message to server
    socket.emit('send_message', messageData)
    setMessage('')
  }
  return (
    <div className="chat-app">
      {!joined ? (
        /* Join Room Card */
        <div className="join-card">
          <h2>Real-Time Chat</h2>
          <p>Practical 12: Socket.IO + React + Node.js</p>
          <form onSubmit={handleJoin} className="join-form">
            <input
              type="text"
              placeholder="Enter your name..."
              value={username}
              onChange={(e) => setUsername(e.target.value)}
              className="chat-input"
              required
            />
            <button type="submit" className="primary-btn">
              Join Chat
            </button>
          </form>
        </div>
      ) : (
        /* Chat Window */
        <div className="chat-box">
          {/* Header */}
          <div className="chat-header">
            <div className="user-status">
              <span className="status-dot"></span>
              <div>
                <h3>Global Chat Room</h3>
                <span className="current-user">Logged in as: <strong>{username}</strong></span>
              </div>
            </div>
            <button
              onClick={() => {
                setJoined(false)
                setMessages([])
              }}
              className="leave-btn"
            >
              Leave
            </button>
          </div>
          {/* Messages Container */}
          <div className="chat-messages">
            {messages.length === 0 ? (
              <p className="no-messages">No messages yet. Say hello!</p>
            ) : (
              messages.map((msg) => {
                const isMe = msg.sender === username
                return (
```

```
              <div
                key={msg.id}
                className={`message-wrapper ${isMe ? 'message-outgoing' : 'message-incoming'}`}
              >
                {!isMe && <span className="sender-name">{msg.sender}</span>}
                <div className="message-bubble">
                  <p className="message-text">{msg.text}</p>
                  <span className="message-time">{msg.time}</span>
                </div>
              </div>
            )
          })
        )}
        <div ref={messagesEndRef} />
      </div>
      {/* Message Input Footer */}
      <form onSubmit={handleSendMessage} className="chat-footer">
        <input
          type="text"
          placeholder="Type your message..."
          value={message}
          onChange={(e) => setMessage(e.target.value)}
          className="chat-input"
        />
        <button type="submit" className="primary-btn">
          Send
        </button>
      </form>
    </div>
  )}
 </div>
)
}
export default App
```

➔ Index.css

```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, Helvetica, Arial, sans-serif;
}
body {
  background-color: #f1f5f9;
  min-height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
  padding: 16px;
}
.chat-app {
  width: 100%;
  max-width: 520px;
}
/* Join Screen */
.join-card {
  background: #ffffff;
  padding: 36px 28px;
  border-radius: 16px;
  box-shadow: 0 10px 25px rgba(0, 0, 0, 0.06);
  text-align: center;
}
```

```
.join-card h2 {
  font-size: 24px;
  color: #0f172a;
  margin-bottom: 6px;
}
.join-card p {
  color: #64748b;
  font-size: 14px;
  margin-bottom: 24px;
}
.join-form {
  display: flex;
  flex-direction: column;
  gap: 12px;
}
/* Chat Box */
.chat-box {
  background: #ffffff;
  border-radius: 16px;
  box-shadow: 0 10px 25px rgba(0, 0, 0, 0.08);
  display: flex;
  flex-direction: column;
  height: 580px;
  overflow: hidden;
}
/* Header */
.chat-header {
  background-color: #ffffff;
  border-bottom: 1px solid #e2e8f0;
  padding: 14px 20px;
  display: flex;
  justify-content: space-between;
  align-items: center;
}
.user-status {
  display: flex;
  align-items: center;
  gap: 10px;
}
.status-dot {
  width: 10px;
  height: 10px;
  background-color: #22c55e;
  border-radius: 50%;
  display: inline-block;
}
.chat-header h3 {
  font-size: 16px;
  color: #0f172a;
}
.current-user {
  font-size: 12px;
  color: #64748b;
}
.leave-btn {
  background-color: #f1f5f9;
  border: 1px solid #e2e8f0;
  color: #64748b;
  padding: 6px 12px;
  border-radius: 6px;
  font-size: 12px;
```

```
  cursor: pointer;
  transition: all 0.2s;
}
.leave-btn:hover {
  background-color: #fee2e2;
  color: #ef4444;
  border-color: #fca5a5;
}
/* Messages List */
.chat-messages {
  flex: 1;
  padding: 20px;
  overflow-y: auto;
  display: flex;
  flex-direction: column;
  gap: 12px;
  background-color: #f8fafc;
}
.no-messages {
  text-align: center;
  color: #94a3b8;
  font-size: 14px;
  margin-top: auto;
  margin-bottom: auto;
}
.message-wrapper {
  display: flex;
  flex-direction: column;
  max-width: 78%;
}
.message-incoming {
  align-self: flex-start;
}
.message-outgoing {
  align-self: flex-end;
}
.sender-name {
  font-size: 11px;
  color: #64748b;
  margin-bottom: 4px;
  margin-left: 6px;
  font-weight: 500;
}
.message-bubble {
  padding: 10px 14px;
  border-radius: 14px;
  display: flex;
  flex-direction: column;
  gap: 4px;
  word-break: break-word;
}
.message-incoming .message-bubble {
  background-color: #ffffff;
  color: #1e293b;
  border: 1px solid #e2e8f0;
  border-bottom-left-radius: 2px;
}
.message-outgoing .message-bubble {
  background-color: #2563eb;
  color: #ffffff;
  border-bottom-right-radius: 2px;
```

```
}
.message-text {
  font-size: 14px;
  line-height: 1.4;
}
.message-time {
  font-size: 10px;
  align-self: flex-end;
  opacity: 0.75;
}
/* Footer / Input */
.chat-footer {
  padding: 14px 18px;
  background-color: #ffffff;
  border-top: 1px solid #e2e8f0;
  display: flex;
  gap: 10px;
}
.chat-input {
  flex: 1;
  padding: 12px 14px;
  border: 1px solid #cbd5e1;
  border-radius: 8px;
  font-size: 14px;
  outline: none;
  transition: border-color 0.2s;
}
.chat-input:focus {
  border-color: #3b82f6;
}
.primary-btn {
  background-color: #2563eb;
  color: #ffffff;
  border: none;
  padding: 12px 20px;
  border-radius: 8px;
  font-size: 14px;
  font-weight: 600;
  cursor: pointer;
  transition: background-color 0.2s;
}
.primary-btn:hover {
  background-color: #1d4ed8;
}
/* Responsive adjustments */
@media (max-width: 580px) {
  body {
    padding: 0;
  }
  .chat-box {
    height: 100vh;
    border-radius: 0;
  }
  .join-card {
    height: 100vh;
    border-radius: 0;
    display: flex;
    flex-direction: column;
    justify-content: center;
  }
}
```

➔ **Main.jsx :**

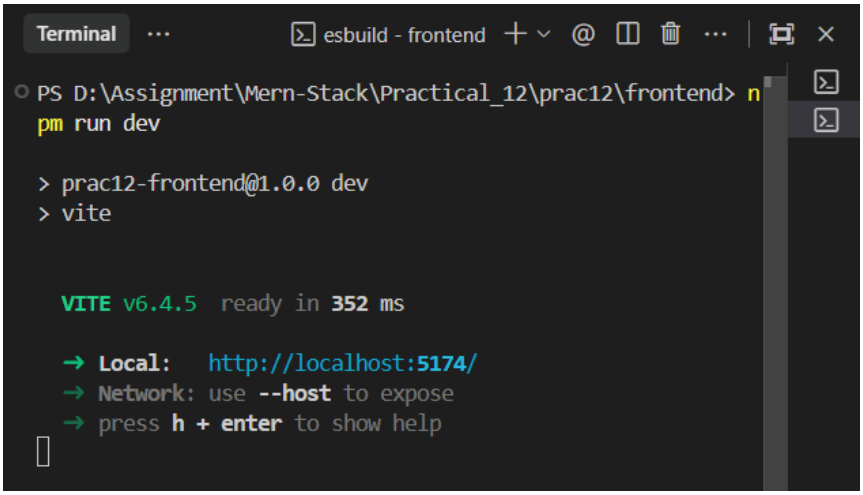
```
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App.jsx'
import './index.css'
ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
)
```

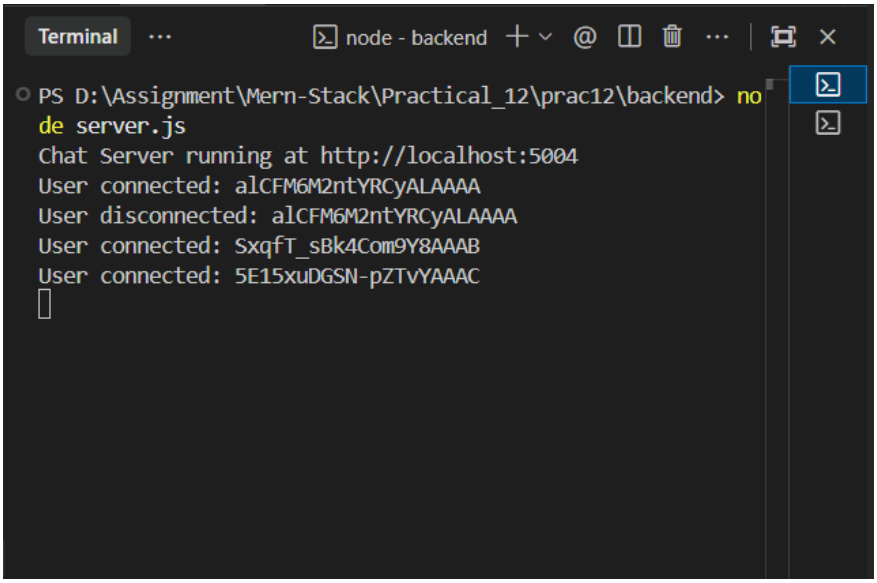
➔ Index.html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Real-Time Chat App - Practical 12</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

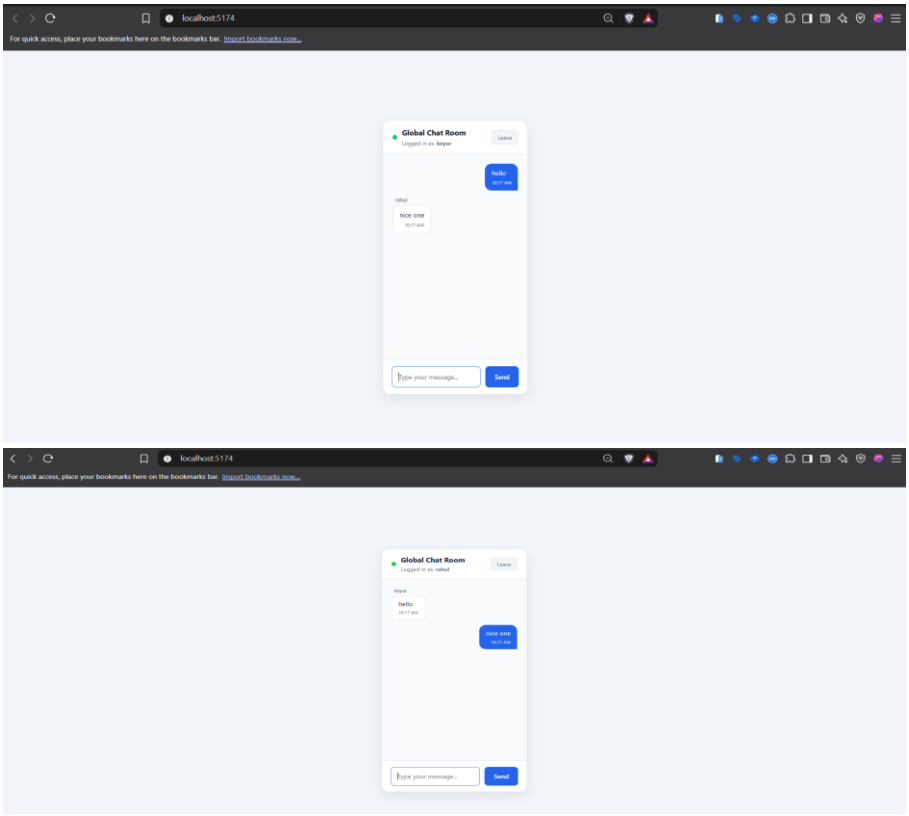
OUTPUT :

1.Terminal Screenshot :





2.Browser OUT of chat :-



Result

Thus, a responsive real-time chat application was successfully developed using React.js, Node.js, Express, and Socket.IO. Bi-directional WebSocket communication was established between the client and server, enabling real-time message broadcasting and reception across multiple browser instances without page refreshing.

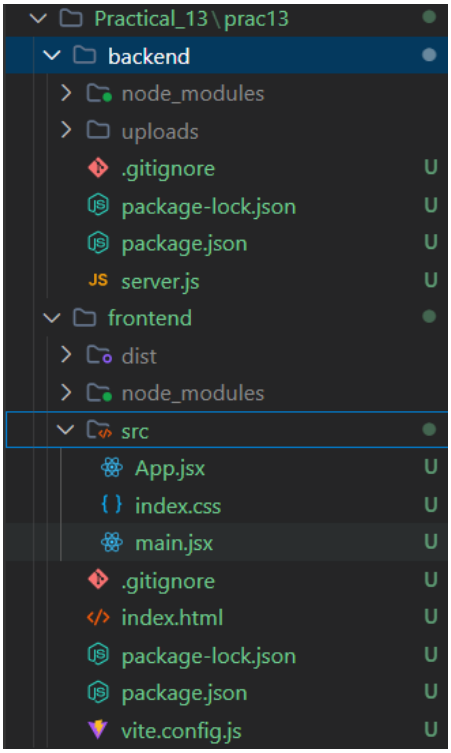
PRACTICAL NO. 13

Mini Project: Design and develop a full-stack industry-oriented MERN application (Document Management System).

Software Requirements

Operating System: Windows 10 / 11
Development Environment: Visual Studio Code
Runtime Environment: Node.js (v18.x or later)
Package Manager: npm (Node Package Manager)
Database: MongoDB & MongoDB Compass (Local or Atlas)
Backend Stack: Node.js, Express.js, Mongoose ODM, Multer (Multipart File Handling), CORS
Frontend Stack: React.js, Vite, Modern Responsive CSS
Testing Tools: Google Chrome, Postman

Folder Structure



Source Code :

➔ **Backend folder :**

Server.js :

```
const express = require('express')
const mongoose = require('mongoose')
const multer = require('multer')
const cors = require('cors')
const path = require('path')
const fs = require('fs')
const app = express()
const PORT = process.env.PORT || 5005
// Middleware
app.use(cors())
app.use(express.json())
// Ensure uploads directory exists
const uploadDir = path.join(__dirname, 'uploads')
if (!fs.existsSync(uploadDir)) {
  fs.mkdirSync(uploadDir, { recursive: true })
}
// Serve uploaded files statically
app.use('/uploads', express.static(uploadDir))
// ==================== 1. MONGODB CONNECTION ====================
const MONGO_URI = 'mongodb://127.0.0.1:27017/documentManagerDB'
mongoose
```

```
  .connect(MONGO_URI)
  .then(() => console.log('MongoDB Connected to documentManagerDB'))
  .catch((err) => console.error('MongoDB Connection Error:', err.message))
// ==================== 2. MONGOOSE SCHEMA & MODEL ====================
const documentSchema = new mongoose.Schema({
  title: {
    type: String,
    required: true,
    trim: true
  },
  category: {
    type: String,
    required: true,
    enum: ['Technical', 'Financial', 'HR', 'Legal', 'General'],
    default: 'General'
  },
  description: {
    type: String,
    trim: true,
    default: ''
  },
  originalName: {
    type: String,
    required: true
  },
  fileName: {
    type: String,
    required: true
  },
  filePath: {
    type: String,
    required: true
  },
  fileSize: {
    type: Number,
    required: true
  },
  mimeType: {
    type: String,
    required: true
  },
  createdAt: {
    type: Date,
    default: Date.now
  }
})
const Document = mongoose.model('Document', documentSchema)
// ==================== 3. MULTER CONFIGURATION ====================
const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, uploadDir)
  },
  filename: (req, file, cb) => {
    const uniqueSuffix = Date.now() + '-' + Math.round(Math.random() * 1e9)
    cb(null, uniqueSuffix + path.extname(file.originalname))
  }
})
const upload = multer({
  storage: storage,
  limits: { fileSize: 15 * 1024 * 1024 } // 15MB limit
})
```

```
// ==================== 4. REST API ROUTES ====================
// Health check
app.get('/', (req, res) => {
  res.json({ message: 'Document Management System API is running' })
})
// POST: Upload document with metadata
app.post('/api/documents/upload', upload.single('file'), async (req, res) => {
  try {
    if (!req.file) {
      return res.status(400).json({ error: 'Please choose a file to upload' })
    }
    const { title, category, description } = req.body
    if (!title || !title.trim()) {
      // Remove uploaded file if validation fails
      fs.unlinkSync(req.file.path)
      return res.status(400).json({ error: 'Document title is required' })
    }
    const newDoc = new Document({
      title: title.trim(),
      category: category || 'General',
      description: description ? description.trim() : '',
      originalName: req.file.originalname,
      fileName: req.file.filename,
      filePath: req.file.path,
      fileSize: req.file.size,
      mimeType: req.file.mimetype
    })
    const savedDoc = await newDoc.save()
    res.status(201).json({
      message: 'Document uploaded and indexed successfully',
      document: savedDoc
    })
  } catch (error) {
    if (req.file && fs.existsSync(req.file.path)) {
      fs.unlinkSync(req.file.path)
    }
    res.status(500).json({ error: error.message })
  }
})
// GET: Fetch documents with optional category and search query
app.get('/api/documents', async (req, res) => {
  try {
    const { category, search } = req.query
    let filter = {}
    if (category && category !== 'All') {
      filter.category = category
    }
    if (search && search.trim()) {
      filter.$or = [
        { title: { $regex: search.trim(), $options: 'i' } },
        { description: { $regex: search.trim(), $options: 'i' } },
        { originalName: { $regex: search.trim(), $options: 'i' } }
      ]
    }
    const documents = await Document.find(filter).sort({ createdAt: -1 })
    res.json(documents)
  } catch (error) {
    res.status(500).json({ error: error.message })
  }
})
// GET: Summary statistics
```

```
app.get('/api/documents/stats', async (req, res) => {
  try {
    const totalDocs = await Document.countDocuments()
    const allDocs = await Document.find({}, 'fileSize category')
    const totalSize = allDocs.reduce((acc, doc) => acc + (doc.fileSize || 0), 0)
    const categories = ['Technical', 'Financial', 'HR', 'Legal', 'General']
    const categoryCounts = {}
    categories.forEach((cat) => {
      categoryCounts[cat] = allDocs.filter((d) => d.category === cat).length
    })
    res.json({
      totalDocs,
      totalSize, // bytes
      categoryCounts
    })
  } catch (error) {
    res.status(500).json({ error: error.message })
  }
})
// GET: Download document file
app.get('/api/documents/download/:id', async (req, res) => {
  try {
    const doc = await Document.findById(req.params.id)
    if (!doc) {
      return res.status(404).json({ error: 'Document not found' })
    }
    if (!fs.existsSync(doc.filePath)) {
      return res.status(404).json({ error: 'File not found on server disk' })
    }
    res.download(doc.filePath, doc.originalName)
  } catch (error) {
    res.status(500).json({ error: error.message })
  }
})
// DELETE: Remove document and delete file from disk
app.delete('/api/documents/:id', async (req, res) => {
  try {
    const doc = await Document.findByIdAndDelete(req.params.id)
    if (!doc) {
      return res.status(404).json({ error: 'Document not found' })
    }
    // Delete file from disk if exists
    if (fs.existsSync(doc.filePath)) {
      fs.unlinkSync(doc.filePath)
    }
    res.json({
      message: 'Document deleted successfully',
      id: req.params.id
    })
  } catch (error) {
    res.status(500).json({ error: error.message })
  }
})
// 404 Fallback
app.use((req, res) => {
  res.status(404).json({ error: 'Route not found' })
})
// ==================== 5. SERVER LISTEN ====================
app.listen(PORT, () => {
  console.log(`Document Management Server running at http://localhost:${PORT}`)
})
```

➔ Frontend Folder :

App.jsx :

```
import React, { useState, useEffect } from 'react'
const API_BASE = 'http://localhost:5005/api/documents'
const CATEGORIES = ['All', 'Technical', 'Financial', 'HR', 'Legal', 'General']
function App() {
  const [documents, setDocuments] = useState([])
  const [stats, setStats] = useState({ totalDocs: 0, totalSize: 0, categoryCounts: {} })
  const [selectedCategory, setSelectedCategory] = useState('All')
  const [searchQuery, setSearchQuery] = useState('')
  const [loading, setLoading] = useState(false)
  const [uploading, setUploading] = useState(false)
  const [statusMessage, setStatusMessage] = useState({ text: '', type: '' })
  // Form state
  const [title, setTitle] = useState('')
  const [category, setCategory] = useState('General')
  const [description, setDescription] = useState('')
  const [file, setFile] = useState(null)
  // Fetch documents
  const fetchDocuments = async () => {
    try {
      setLoading(true)
      const queryParams = new URLSearchParams()
      if (selectedCategory !== 'All') queryParams.append('category', selectedCategory)
      if (searchQuery.trim()) queryParams.append('search', searchQuery.trim())
      const res = await fetch(`${API_BASE}?${queryParams.toString()}`)
      const data = await res.json()
      setDocuments(Array.isArray(data) ? data : [])
    } catch (err) {
      console.error('Error fetching documents:', err)
    } finally {
      setLoading(false)
    }
  }
  // Fetch stats
  const fetchStats = async () => {
    try {
      const res = await fetch(`${API_BASE}/stats`)
      const data = await res.json()
      setStats(data)
    } catch (err) {
      console.error('Error fetching stats:', err)
    }
  }
  useEffect(() => {
    fetchDocuments()
    fetchStats()
  }, [selectedCategory])
  // Trigger search on submit or debounce
  const handleSearchSubmit = (e) => {
    e.preventDefault()
    fetchDocuments()
  }
  // Handle document upload
  const handleUpload = async (e) => {
    e.preventDefault()
    if (!file) {
      setStatusMessage({ text: 'Please select a file to upload.', type: 'error' })
      return
    }
    try {
```

```
    setUploading(true)
    const formData = new FormData()
    formData.append('title', title)
    formData.append('category', category)
    formData.append('description', description)
    formData.append('file', file)
    const res = await fetch(`${API_BASE}/upload`, {
      method: 'POST',
      body: formData
    })
    const data = await res.json()
    if (res.ok) {
      setStatusMessage({ text: 'Document uploaded successfully!', type: 'success' })
      setTitle('')
      setCategory('General')
      setDescription('')
      setFile(null)
      // Reset file input in DOM
      e.target.reset()
      fetchDocuments()
      fetchStats()
    } else {
      setStatusMessage({ text: data.error || 'Upload failed', type: 'error' })
    }
  } catch (err) {
    setStatusMessage({ text: 'Error connecting to server', type: 'error' })
  } finally {
    setUploading(false)
    setTimeout(() => setStatusMessage({ text: '', type: '' }), 4000)
  }
}
// Handle document delete
const handleDelete = async (id) => {
  if (!window.confirm('Are you sure you want to delete this document?')) return
  try {
    const res = await fetch(`${API_BASE}/${id}`, { method: 'DELETE' })
    if (res.ok) {
      setStatusMessage({ text: 'Document deleted successfully!', type: 'success' })
      fetchDocuments()
      fetchStats()
    }
  } catch (err) {
    console.error('Failed to delete document:', err)
  }
}
// Helper: Format bytes to human readable string
const formatBytes = (bytes) => {
  if (!bytes || bytes === 0) return '0 B'
  const k = 1024
  const sizes = ['B', 'KB', 'MB', 'GB']
  const i = Math.floor(Math.log(bytes) / Math.log(k))
  return parseFloat((bytes / Math.pow(k, i)).toFixed(2)) + ' ' + sizes[i]
}
// Helper: File Type Badge
const getFileBadge = (mime) => {
  if (mime.includes('pdf')) return { label: 'PDF', color: 'badge-red' }
  if (mime.includes('word') || mime.includes('officedocument')) return { label: 'DOC', color: 'badge-blue' }
  if (mime.includes('image')) return { label: 'IMG', color: 'badge-green' }
  return { label: 'FILE', color: 'badge-purple' }
}
return (
```

```
<div className="dms-app">
 {/* Navbar */}
 <header className="navbar">
  <div className="nav-brand">
   <div className="brand-icon">📁</div>
   <div>
    <h1>DocuVault DMS</h1>
    <span className="subtitle">Practical 13: Full-Stack MERN Application</span>
   </div>
  </div>
  <form onSubmit={handleSearchSubmit} className="search-form">
   <input
    type="text"
    placeholder="Search documents by name or keyword..."
    value={searchQuery}
    onChange={(e) => setSearchQuery(e.target.value)}
    className="search-input"
   />
   <button type="submit" className="btn-search">Search</button>
  </form>
 </header>
 {/* Main Content Container */}
 <main className="container">
  {/* Status Notification */}
  {statusMessage.text && (
   <div className={`alert-box alert-${statusMessage.type}`}>
    {statusMessage.text}
   </div>
  )}
  {/* Dashboard Analytics Cards */}
  <section className="stats-grid">
   <div className="stat-card">
    <span className="stat-label">Total Documents</span>
    <span className="stat-value">{stats.totalDocs || 0}</span>
   </div>
   <div className="stat-card">
    <span className="stat-label">Total Storage Used</span>
    <span className="stat-value">{formatBytes(stats.totalSize)}</span>
   </div>
   <div className="stat-card">
    <span className="stat-label">Selected Category</span>
    <span className="stat-value">{selectedCategory}</span>
   </div>
  </section>
  {/* Dashboard Layout: Left Form + Right Library */}
  <div className="dashboard-grid">
   {/* Upload Document Panel */}
   <section className="panel upload-panel">
    <h2>Upload New Document</h2>
    <form onSubmit={handleUpload} className="upload-form">
     <div className="form-group">
      <label>Document Title *</label>
      <input
       type="text"
       placeholder="e.g. Q3 Financial Report"
       value={title}
       onChange={(e) => setTitle(e.target.value)}
       required
      />
     </div>
     <div className="form-group">
```

```
      <label>Category *</label>
      <select value={category} onChange={(e) => setCategory(e.target.value)}>
        {CATEGORIES.filter((c) => c !== 'All').map((cat) => (
          <option key={cat} value={cat}>{cat}</option>
        ))}
      </select>
    </div>
    <div className="form-group">
      <label>Description</label>
      <textarea
        rows="3"
        placeholder="Brief summary of document content..."
        value={description}
        onChange={(e) => setDescription(e.target.value)}
      />
    </div>
    <div className="form-group">
      <label>Select File (PDF, DOCX, TXT, IMG) *</label>
      <input
        type="file"
        onChange={(e) => setFile(e.target.files[0])}
        required
      />
    </div>
    <button type="submit" disabled={uploading} className="btn-upload">
      {uploading ? 'Uploading...' : 'Upload Document'}
    </button>
  </form>
</section>
{/* Document Library Panel */}
<section className="panel library-panel">
  <div className="library-header">
    <h2>Document Library ({documents.length})</h2>
    {/* Category Filter Pills */}
    <div className="category-pills">
      {CATEGORIES.map((cat) => (
        <button
          key={cat}
          onClick={() => setSelectedCategory(cat)}
          className={`pill-btn ${selectedCategory === cat ? 'active' : ''}`}
        >
          {cat}
        </button>
      ))}
    </div>
  </div>
  {/* Document Cards */}
  {loading ? (
    <p className="loading-state">Loading documents...</p>
  ) : documents.length === 0 ? (
    <div className="empty-state">
      <p>No documents found matching the criteria.</p>
      <small>Upload your first document using the form on the left!</small>
    </div>
  ) : (
    <div className="documents-list">
      {documents.map((doc) => {
        const badge = getFileBadge(doc.mimeType)
        return (
          <div key={doc._id} className="document-card">
            <div className="doc-header">
```

```
                  <span className={`file-badge ${badge.color}`}>{badge.label}</span>
                  <div className="doc-main-info">
                    <h3 className="doc-title">{doc.title}</h3>
                    <span className="doc-category">{doc.category}</span>
                  </div>
                </div>
                {doc.description && <p className="doc-desc">{doc.description}</p>}
                <div className="doc-meta">
                  <span>Original: {doc.originalName}</span>
                  <span>Size: {formatBytes(doc.fileSize)}</span>
                  <span>Uploaded: {new Date(doc.createdAt).toLocaleDateString()}</span>
                </div>
                <div className="doc-actions">
                  <a
                    href={`${API_BASE}/download/${doc._id}`}
                    className="btn-download"
                    download
                  >
                    Download
                  </a>
                  <button
                    onClick={() => handleDelete(doc._id)}
                    className="btn-delete"
                  >
                    Delete
                  </button>
                </div>
              </div>
            )
          })}
        </div>
      )}
    </section>
  </div>
 </main>
</div>
 )
}
export default App
```

index.css :

```
 * {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, Helvetica, Arial, sans-serif;
}
body {
  background-color: #f8fafc;
  color: #1e293b;
  min-height: 100vh;
}
.dms-app {
  display: flex;
  flex-direction: column;
  min-height: 100vh;
}
/* Navbar */
.navbar {
  background-color: #ffffff;
  border-bottom: 1px solid #e2e8f0;
  padding: 16px 28px;
```

```
  display: flex;
  justify-content: space-between;
  align-items: center;
  flex-wrap: wrap;
  gap: 16px;
}
.nav-brand {
  display: flex;
  align-items: center;
  gap: 12px;
}
.brand-icon {
  font-size: 28px;
}
.nav-brand h1 {
  font-size: 20px;
  color: #0f172a;
  font-weight: 700;
}
.subtitle {
  font-size: 12px;
  color: #64748b;
}
.search-form {
  display: flex;
  gap: 8px;
  flex: 1;
  max-width: 420px;
}
.search-input {
  flex: 1;
  padding: 10px 14px;
  border: 1px solid #cbd5e1;
  border-radius: 8px;
  font-size: 14px;
  outline: none;
}
.search-input:focus {
  border-color: #4f46e5;
}
.btn-search {
  background-color: #4f46e5;
  color: #ffffff;
  border: none;
  padding: 10px 16px;
  border-radius: 8px;
  font-size: 13px;
  font-weight: 600;
  cursor: pointer;
}
/* Container */
.container {
  max-width: 1200px;
  margin: 0 auto;
  padding: 24px 20px;
  width: 100%;
  display: flex;
  flex-direction: column;
  gap: 20px;
}
/* Notifications */
```

```
.alert-box {
  padding: 12px 18px;
  border-radius: 8px;
  font-size: 14px;
  text-align: center;
}
.alert-success {
  background-color: #dcfce7;
  color: #15803d;
  border: 1px solid #bbf7d0;
}
.alert-error {
  background-color: #fee2e2;
  color: #b91c1c;
  border: 1px solid #fecaca;
}
/* Analytics Stats Grid */
.stats-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 16px;
}
.stat-card {
  background: #ffffff;
  padding: 20px;
  border-radius: 12px;
  border: 1px solid #e2e8f0;
  display: flex;
  flex-direction: column;
  gap: 6px;
  box-shadow: 0 2px 4px rgba(0, 0, 0, 0.02);
}
.stat-label {
  font-size: 13px;
  color: #64748b;
  font-weight: 500;
}
.stat-value {
  font-size: 24px;
  color: #0f172a;
  font-weight: 700;
}
/* Dashboard Two-Column Grid */
.dashboard-grid {
  display: grid;
  grid-template-columns: 360px 1fr;
  gap: 24px;
  align-items: flex-start;
}
.panel {
  background: #ffffff;
  border: 1px solid #e2e8f0;
  border-radius: 14px;
  padding: 24px;
  box-shadow: 0 2px 6px rgba(0, 0, 0, 0.02);
}
.panel h2 {
  font-size: 18px;
  color: #0f172a;
  margin-bottom: 18px;
}
```

```
/* Upload Form */
.upload-form {
  display: flex;
  flex-direction: column;
  gap: 16px;
}
.form-group {
  display: flex;
  flex-direction: column;
  gap: 6px;
}
.form-group label {
  font-size: 13px;
  font-weight: 600;
  color: #334155;
}
.form-group input,
.form-group select,
.form-group textarea {
  padding: 10px 12px;
  border: 1px solid #cbd5e1;
  border-radius: 8px;
  font-size: 14px;
  outline: none;
  background-color: #ffffff;
}
.form-group input:focus,
.form-group select:focus,
.form-group textarea:focus {
  border-color: #4f46e5;
}
.btn-upload {
  background-color: #4f46e5;
  color: #ffffff;
  border: none;
  padding: 12px;
  border-radius: 8px;
  font-size: 14px;
  font-weight: 600;
  cursor: pointer;
  transition: background-color 0.2s;
  margin-top: 6px;
}
.btn-upload:hover {
  background-color: #4338ca;
}
.btn-upload:disabled {
  opacity: 0.6;
  cursor: not-allowed;
}
/* Document Library */
.library-header {
  display: flex;
  flex-direction: column;
  gap: 14px;
  margin-bottom: 20px;
}
.category-pills {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
```

```
}
.pill-btn {
  background: #f1f5f9;
  border: 1px solid #e2e8f0;
  color: #475569;
  padding: 6px 14px;
  border-radius: 20px;
  font-size: 12px;
  font-weight: 500;
  cursor: pointer;
  transition: all 0.2s;
}
.pill-btn:hover {
  background-color: #e2e8f0;
}
.pill-btn.active {
  background-color: #4f46e5;
  color: #ffffff;
  border-color: #4f46e5;
}
.loading-state,
.empty-state {
  text-align: center;
  padding: 40px 20px;
  color: #94a3b8;
  font-size: 14px;
}
/* Documents List / Cards */
.documents-list {
  display: flex;
  flex-direction: column;
  gap: 14px;
}
.document-card {
  border: 1px solid #e2e8f0;
  border-radius: 12px;
  padding: 18px;
  background: #ffffff;
  transition: transform 0.15s, box-shadow 0.15s;
  display: flex;
  flex-direction: column;
  gap: 12px;
}
.document-card:hover {
  transform: translateY(-2px);
  box-shadow: 0 6px 12px rgba(0, 0, 0, 0.04);
}
.doc-header {
  display: flex;
  align-items: center;
  gap: 12px;
}
.file-badge {
  padding: 6px 10px;
  border-radius: 6px;
  font-size: 11px;
  font-weight: 700;
  text-transform: uppercase;
}
.badge-red { background-color: #fee2e2; color: #dc2626; }
.badge-blue { background-color: #dbeafe; color: #2563eb; }
```

```
.badge-green { background-color: #dcfce7; color: #16a34a; }
.badge-purple { background-color: #f3e8ff; color: #9333ea; }
.doc-main-info {
  display: flex;
  flex-direction: column;
}
.doc-title {
  font-size: 16px;
  color: #0f172a;
  font-weight: 600;
}
.doc-category {
  font-size: 12px;
  color: #64748b;
}
.doc-desc {
  font-size: 13px;
  color: #475569;
  line-height: 1.4;
}
.doc-meta {
  display: flex;
  flex-wrap: wrap;
  gap: 14px;
  font-size: 12px;
  color: #94a3b8;
  border-top: 1px dashed #f1f5f9;
  padding-top: 8px;
}
.doc-actions {
  display: flex;
  gap: 10px;
  margin-top: 4px;
}
.btn-download {
  background-color: #f0fdf4;
  border: 1px solid #bbf7d0;
  color: #16a34a;
  text-decoration: none;
  font-size: 12px;
  font-weight: 600;
  padding: 6px 14px;
  border-radius: 6px;
  transition: all 0.2s;
}
.btn-download:hover {
  background-color: #dcfce7;
}
.btn-delete {
  background-color: #fef2f2;
  border: 1px solid #fecaca;
  color: #dc2626;
  font-size: 12px;
  font-weight: 600;
  padding: 6px 14px;
  border-radius: 6px;
  cursor: pointer;
  transition: all 0.2s;
}
.btn-delete:hover {
  background-color: #fee2e2;
```

```
}
/* Responsive Breakpoints */
@media (max-width: 900px) {
  .dashboard-grid {
    grid-template-columns: 1fr;
  }
  .navbar {
    flex-direction: column;
    align-items: stretch;
  }
  .search-form {
    max-width: 100%;
  }
}
```

main.jsx :

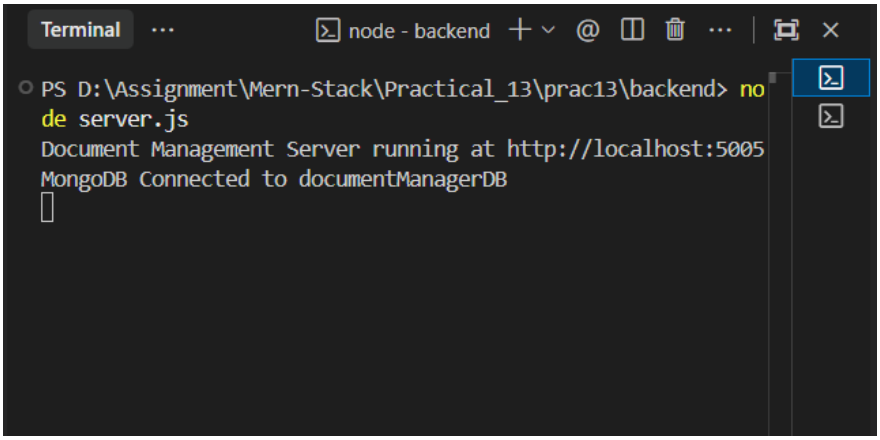
```
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App.jsx'
import './index.css'
ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
)
```

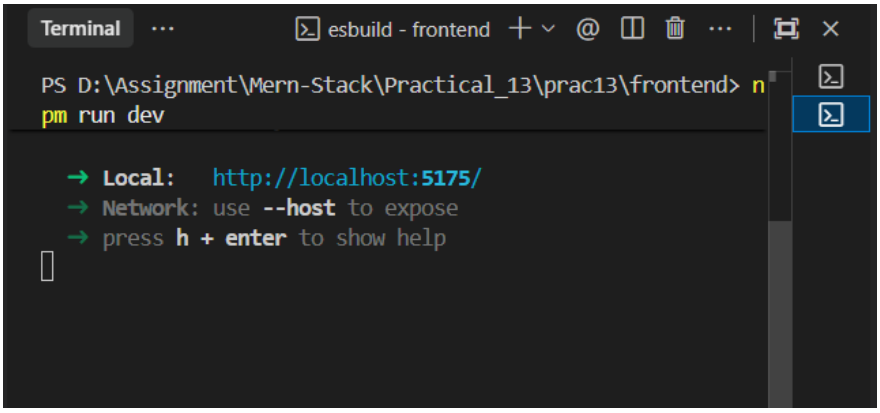
Index.html :

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>DocuVault - Document Management System</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

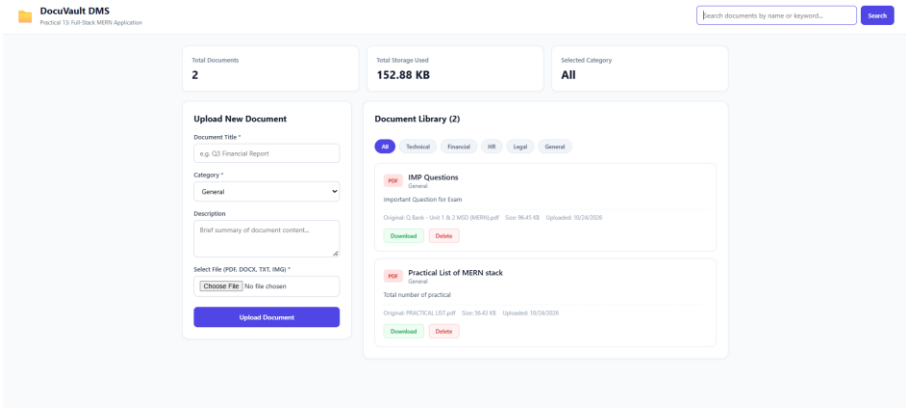
OUTPUT :

1.Terminal Screensot :

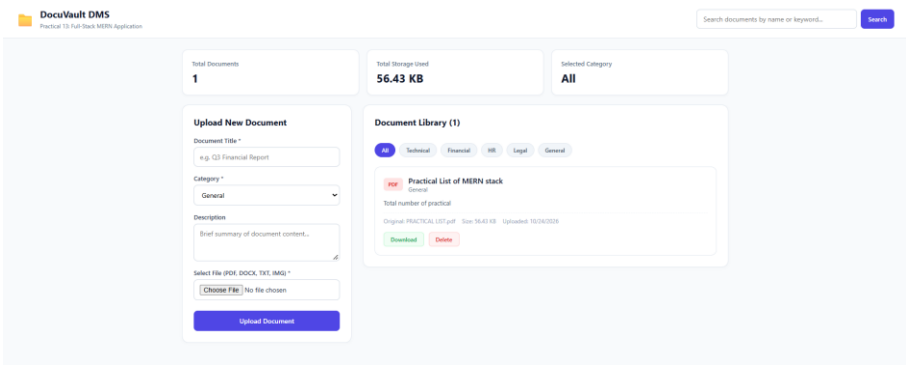




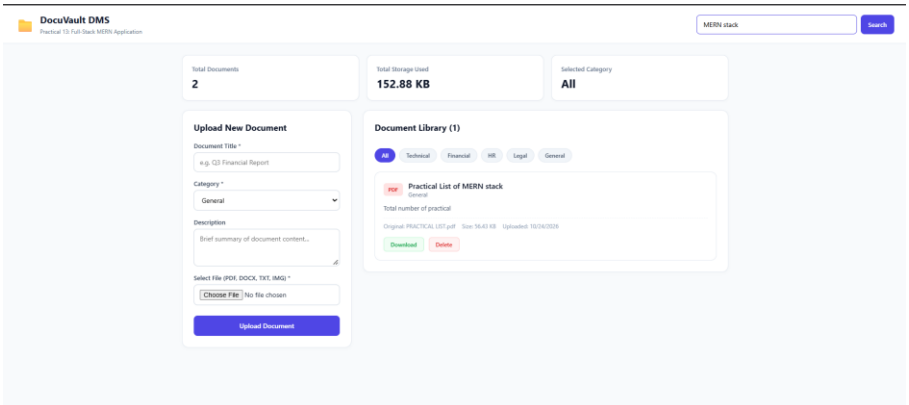
2.Brrowser Output Before filling anything :



3.Browser output after Filling Fields :



4.perfom search :



Result

Thus, a full-stack industry-oriented Document Management System (DMS) was successfully designed and implemented using the MERN stack (MongoDB, Express.js, React.js, and Node.js) with Multer file storage. The system demonstrated end-to-end full-stack capabilities, including multipart document uploads, database indexing, real-time search, category filtering, file downloading, dynamic analytics calculations, and a responsive frontend interface.